ChaosFormer: Hardware-Bound Edge Transformer Model Obfuscation for Intellectual Property Protection

PEICHUN HUA, School of Data Science, The Chinese University of Hong Kong, Shenzhen, P.R. China
YUE ZHENG*, School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen, P.R. China

Building a production-level deep learning (DL) model requires substantial data, computational resources, and expert knowledge, making it a valuable asset vulnerable to intellectual property (IP) theft. Apart from violating commercial interests, the stolen model facilitates adversarial attacks and reduces the cost of building malicious applications. Thus, it is urgent today to protect DL models from leakage and theft. Existing solutions, such as watermarking and fingerprinting, provide only passive ownership verification after model theft. Instead, we explore an active protection method for edge-deployed transformer models. This paper presents `ChaosFormer`, a hardware-bound obfuscation framework that binds model executions to specific hardware using physical unclonable functions (PUF). We introduce a base permutation scheme for efficiency and a secure extension (`ChaosFormer-S`) incorporating diffusion for enhanced security. By obfuscating model weights with keys derived from PUF, `ChaosFormer` ensures that only authorized users can run the model on authorized machines, thereby actively preventing model misuse. We provide formal proofs for the system's chaotic dynamics and security (Single-Point Indistinguishability). Experiments show that unauthorized execution collapses encrypted models from 77.1–86.4% ImageNet accuracy to near-random levels of 0.07–0.15%; under retraining attacks, the base scheme substantially reduces stolen-model utility, while `ChaosFormer-S` further lowers retrained ViT-B performance below same-architecture random initialization, e.g., 27.36% vs. 36.19% on ImageNet and 44.54% vs. 52.95% on DomainNet. In addition, our proposed method does not require additional retraining and incurs little inference overhead. This approach offers a low-cost, effective solution for the secure deployment of DL models at the edge. Our source code is publicly available at https://github.com/sarendis56/ChaosFormer-IP-Protection.

CCS Concepts: • **Security and privacy → Software and application security**; **Hardware-based security protocols**; • **Computing methodologies** → *Neural networks*.

Additional Key Words and Phrases: Intellectual Property Protection, Transformer Models, Physical Unclonable Functions, Adversarial Machine Learning, Format-Preserving Encryption

ACM Reference Format:
Peichun Hua and Yue Zheng. 2025. ChaosFormer: Hardware-Bound Edge Transformer Model Obfuscation for Intellectual Property Protection. *ACM Trans. Des. Autom. Electron. Syst.* 0, 0, Article 0 (January 2025), 36 pages. https://doi.org/XXXXXXX.XXXXXXX

*Corresponding author.

Authors' Contact Information: Peichun Hua, School of Data Science, The Chinese University of Hong Kong, Shenzhen, Shenzhen, P.R. China, peichunhua@link.cuhk.edu.cn; Yue Zheng, School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen, Shenzhen, P.R. China, zhengyue@cuhk.edu.cn.

ACM 1557-7309/2025/1-ART0
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Deep neural networks (DNNs) have become indispensable in many applications, driving remarkable achievements in computer vision, natural language processing, and more. Among them, *transformer* [105] has emerged as a predominant architecture across various tasks due to its superior performance and scalability. These models typically demand extensive data collection and massive computational resources for training, making well-tuned weights a valuable intellectual property (IP) asset.

Deploying transformers at the edge (e.g., in smartphones, IoT devices, or embedded systems) can bring substantial benefits, such as low latency, reduced cloud dependence, powerful functionality without internet connectivity, and enhanced user privacy. However, it also introduces critical security challenges. Once a transformer model is locally stored on a device, malicious actors may seek to *steal* and *misuse* its parameters [20]. With access to these parameters, they could create harmful applications with minimal effort or launch adversarial attacks targeting the original model.

Although various IP protection strategies have been proposed for neural networks, ranging from watermarking [1, 41, 104] and fingerprinting [116, 129] to trusted execution environments (TEEs) [59, 127], they are often ill-suited for transformers deployed at the edge. Watermarking and fingerprinting provide only a *passive* ownership verification mechanism rather than proactively preventing model theft. Specifically, watermarking typically requires significant retraining efforts to embed ownership information, while fingerprinting relies on extracting unique intrinsic identifiers from the target model to prove prior possession after a theft has occurred. Furthermore, these passive methods generally lack formal security guarantees and remain fragile against removal and spoofing attacks [15, 117]. Meanwhile, TEE-based approaches [59, 127] introduce substantial runtime overhead and memory constraints, which are particularly problematic for large transformer models.

Consequently, there is a critical need for a *lightweight* and *proactive* IP protection method that can safeguard *on-device* transformers without requiring retraining or complicated hardware support. Specifically, we seek a solution that: (1) ensures only authorized hardware can correctly run the model (sec. 5.1); (2) incurs minimal inference overhead (sec. 5.3); (3) does not require retraining the original model (sec. 4.2); (4) does not negatively impact model performance, as previous watermark methods did (sec. 5.1); (5) maintains platform independence by operating solely on model weights without using framework-specific features (sec. 4.2); and (6) remains robust even if attackers have partial access to the training data, full knowledge of the network architecture, and access to public pre-trained reference models (sec. 5.2.1).

In this paper, we propose `ChaosFormer`, a hardware-bound encryption framework designed for the proactive IP protection of transformers. `ChaosFormer` *binds* the transformer model's execution to a specific hardware device in order to prevent unauthorized usage — even if an attacker gains white-box access to the stored parameters. Our approach leverages (1) Physical Unclonable Functions (PUF) to bind the model to the target device. Each target device integrates a PUF that reliably derives a unique cryptographic key based on uncontrollable silicon variations. This key is never stored on the device, preventing attackers from reliably extracting it from the memory; and (2) reversible weight obfuscation combined with highly efficient permutation to effectively scramble the weights in a reversible yet highly disruptive manner. We proffer two schemes: the base `ChaosFormer` uses efficient permutation for low latency, while `ChaosFormer-S` adds a diffusion step to ensure statistical indistinguishability from random noise. The correct PUF-based key is indispensable for reconstructing the original layer weights on the authorized device. Our main contributions are as follows:

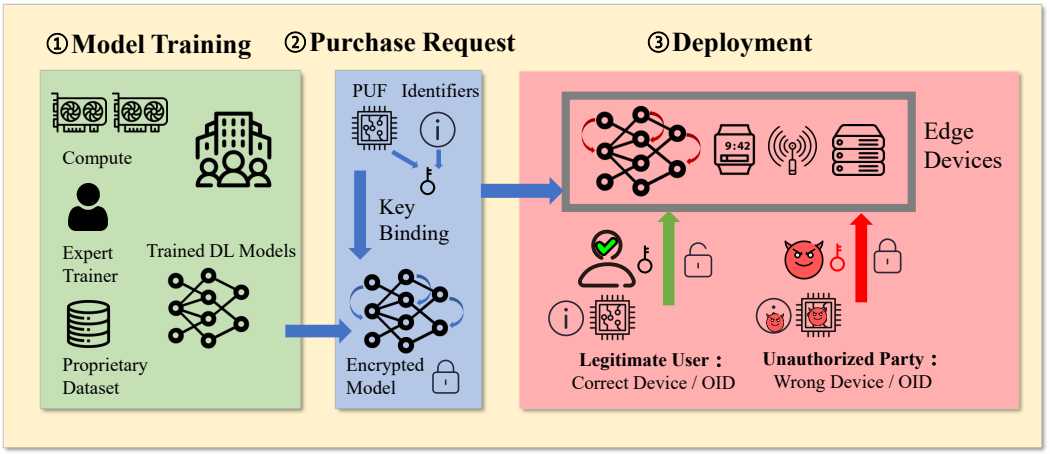


Fig. 1. Overview of our proposed framework, which consists of three stages. (1) Model Training: The trainer trains the model using proprietary datasets and compute resources. (2) Purchase Request: The model is encrypted and bound to the purchaser's device using PUF-derived keys, ensuring that the model can only be deployed on authorized hardware. (3) Deployment: The model remains secure against on-device extraction and server breaches, as unauthorized decryption is infeasible without the device-specific PUF keys.

- A novel proactive IP protection framework that locks transformer models so that they can execute correctly only on authorized devices with valid PUF-derived keys.
- We provide formal security guarantees, proving that our permutation exhibits chaotic behavior (avalanche effect) and that `ChaosFormer-S` satisfies Single-Point Indistinguishability (SPI).
- Extensive experiments demonstrate that attackers cannot recover the model; specifically, we show that the stolen model provides no utility benefit over training a distinct model from scratch.
- The inference overhead is minimal, and our obfuscation process applies to various transformer architectures, making it practical for real-world edge deployments.

The rest of the paper is organized as follows. Sec. 2 introduces background on transformer architectures, Arnold's Cat Map, and PUF. Sec. 3 details our threat model. Our IP protection framework is presented in sec. 4. Sec. 5 describes our experimental evaluation and comparisons. We discuss related work in sec. 6 and conclude in sec. 7.

2 Preliminaries

2.1 Transformer

Transformers are DL models with strong capabilities in language modeling [11], computer vision [37], time series [111], code generation [46], and software security [24, 123]. A typical transformer [105] consists of a series of transformer layers stacked together. Each layer has two main components: *multi-head self-attention* and a *feedforward network (FFN)*. Fig. 2 illustrates a standard transformer architecture, with our targets highlighted in deep blue.

Multi-Head Self-Attention. Given input embeddings $X \in \mathbb{R}^{n \times d}$, the Query (Q), Key (K) and Value (V) vectors are computed as $Q = XW^Q$, $K = XW^K$, and $V = XW^V$, and then the attention scores are computed via $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})\, V$, where W^Q, W^K, W^V are three learnable weight matrices and d_k denotes the dimensionality of Q and K. To better capture multiple types of relationships in the sequence at the same time, we typically split the attention mechanism

into h heads, compute attention individually, and aggregate with a W^O matrix: $\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$, where $\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$.

Feedforward Network. The FFN applies two linear layers separated by a GELU activation: $\text{FFN}(X) = \text{GELU}(XW_1 + b_1)\, W_2 + b_2$. Residual connections and layer normalization follow each sub-layer. Our scheme focuses on encrypting the multi-head attention and FFN weights, which together account for the bulk of the parameters in a transformer model.

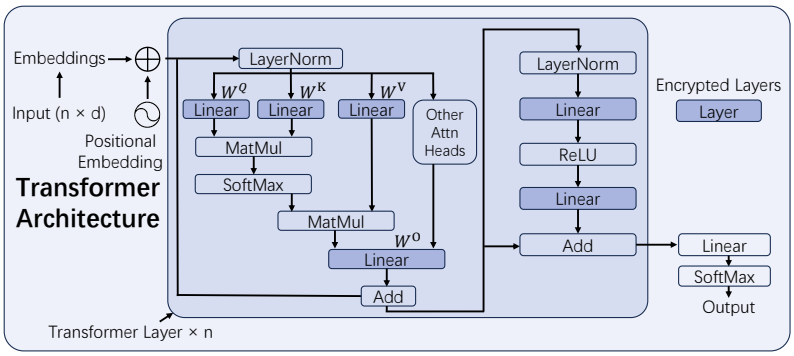


Fig. 2. Block diagram of a standard transformer layer. The components targeted by our scheme are denoted in deep blue (see later in sec. 4.2), which account for most of the parameters in the transformer layer.

2.2 Arnold's Cat Map

Arnold's Cat Map (ACM) is a chaotic transposition technique originally introduced into image cryptography by Fridrich [29] as a more efficient alternative to general-purpose block ciphers for media data, exploiting the large capacity and high redundancy of images. Fridrich's original construction interleaves two phases per round: a *permutation* phase that shuffles pixel positions using a chaotic map and a *diffusion* phase that propagates information across the image to break local correlations. Since then, the ACM family has been adapted to medical image protection [13], voice communication encryption [25], and a series of generalizations enlarging the key space and dimensionality [101, 113, 125]. Lin et al. [61] were the first to repurpose ACM for neural-network weight obfuscation, applying it to CNN and RNN weights with a restricted two-parameter form. We build on this lineage but adapt it specifically to transformers: the attention projection matrices are square and naturally fit the ACM domain, while the FFN matrices are non-square and require a separate permutation primitive (see sec. 4.2). To ensure robust protection that satisfies formal privacy guarantees against adversaries who possess a public reference model (sec. 3), we further extend the standard ACM permutation with a value-diffusion step inspired by the original Fridrich construction.

Permutation Phase. The ACM scrambles the position of an element at coordinates (x_0, y_0) in an $S \times S$ matrix to a new position (x_n, y_n) over n iterations. We adopt the restricted form proposed by Lin et al. [61] for its efficiency. The coordinate transformation and its reversible decryption are defined compactly as:

$$\underbrace{\begin{bmatrix} x_n \\ y_n \end{bmatrix} = A^n \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}}_{\text{Encryption}}, \quad \underbrace{\begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = A^{-n} \begin{bmatrix} x_n \\ y_n \end{bmatrix}}_{\text{Decryption}} \pmod{S} \tag{1}$$

where $A = \begin{bmatrix} 1 & p \\ q & pq+1 \end{bmatrix}$ is the transformation matrix with parameters $p, q \in \mathbb{Z}^+$. Since $\det(A) = 1$, the map is area-preserving and fully reversible.

Diffusion Phase (Optional). Pure permutation affects only the positional correlations but leaves the statistical distribution of values unchanged. To resist statistical analysis, we introduce an optional diffusion step. Let $v_{(x,y)}$ denote the parameter value after the permutation phase at position (x, y). The diffusion operation then applies a modulo addition using a pseudo-random mask $\mathcal{M}$ derived from the diffusion key:

$$v'_{(x,y)} = v_{(x,y)} + \mathcal{M}_{(x,y)} \pmod{L} \tag{2}$$

where $v'_{(x,y)}$ is the new value after diffusion and L represents the domain size of the weights (ensuring the output remains valid format-wise). While conceptually sequential, these two phases are executed via kernel fusion in practice (see sec. 4.2.3). This ensures that even if the permutation key is leaked, the underlying weight values remain statistically obfuscated.

Our scheme, with or without the optional diffusion step, does not attempt to limit the encryption to a small portion of weights [110, 130], which were shown in [94] to expose a statistical vulnerability that facilitates attacks.

2.3 Physical Unclonable Functions (PUF)

PUF is a hardware security primitive designed based on uncontrollable and unpredictable manufacturing process variations in circuit design [10, 92]. A PUF can be mathematically viewed as an irreversible probabilistic function that produces a random bit string (the *response*) given an external *challenge* to the circuit:

$$PUF(ch) : \ r \quad (ch \in Ch, r \in R). \tag{3}$$

where Ch is the set of all external *challenges*, and R describes the *responses* produced by the PUF. The challenge-to-response mapping of a specific PUF is unique and irreproducible, just as no two people have identical fingerprints due to natural biological variations, making PUF a promising technique for generating device-specific keys and hardware identifiers.

In practice, for better security and robustness against attacks and random errors, a technique called controlled PUF (CPUF) is often adopted such that PUFs can only be accessed via an algorithm that is physically bound to the PUF in an inseparable way [32]. In particular, the hardware designer can create the package such that its existence has a significant influence on the PUF measurement, and any efforts to tamper with the PUF or probe PUF responses will change the behavior and destroy the PUF immediately. Meanwhile, the user or application engages with the PUF through a controlled and non-bypassable interface to reach the shared secret. In this work, it is assumed that the hardware provider incorporates such a control mechanism within the PUF design, ensuring that a physical adversary is unable to directly probe the PUF responses.

Realizations on edge accelerators. Although `ChaosFormer` is agnostic to the underlying PUF construction, it is useful to outline the realistic options for edge accelerators so that the hardware-bound claim is concrete. For example, Arbiter PUFs [91] exploit the propagation-delay variation between identically routed signal paths and are widely deployed on FPGAs because they reuse standard logic resources. SRAM PUFs [39] read out the random power-on state of uninitialized SRAM cells and are attractive for microcontroller-class edge devices, where SRAM is already on-chip. DRAM-based variants [51, 98] reuse main memory either through capacitor decay or through deliberately violated row-activation timing, which makes them well-suited to accelerators that expose DRAM but cannot afford additional silicon. For the purposes of ChaosFormer, what matters is only that (i) the chosen PUF supplies enough entropy for a $\geq$ 128-bit response after fuzzy extractor post-processing [10, 19, 50], and (ii) the response is reliably reproducible across the

device's expected operating envelope, which can be ensured using the fuzzy extractor mechanism with error correction incorporated [10]. In our proposed scheme, we do not impose constraints on the specific type of PUF utilized. Any PUF families that satisfy these properties can be adopted for `ChaosFormer` with no algorithmic change. Thanks to the PUF, the obfuscation keys are effectively generated only during model inference on the specific device and are concealed when the model is in transit or at rest.

3 Threat Model

We consider a deployment scenario where transformer models are executed on edge devices. We assume the hardware root-of-trust (e.g., PUF) functions correctly and is tamper-resistant against physical extraction, ensuring that the device-specific decryption keys are never exposed to the user or stored in non-volatile memory. We exclude invasive hardware attacks (e.g., decapping) and direct side-channel attacks on the PUF circuitry itself from our threat model, as these require specialized equipment and are orthogonal to the proposed algorithmic protection. Machine learning-based modeling attacks are also not applicable, as raw PUF challenge-response pairs (CRPs) are not extractable in our framework for substitute PUF model training. However, we assume a powerful adversary with *white-box access* to the encrypted model file. Specifically, the attacker possesses:

- **Model Knowledge:** Full access to the encrypted parameters and model architecture.
- **Data:** Access to a subset of the original training data and full downstream datasets.
- **Computational Resources:** Sufficient power to attempt model recovery via retraining, fine-tuning, or brute-force key search.
- **Timing Side-Channel Information:** The ability to observe inference latency, potentially revealing which layers are encrypted via timing analysis (addressed in sec. 5.2.2).

Tab. 1 summarizes the attacker capabilities admitted under our threat model. Indirect black-box model extraction [75, 76, 103], which queries a deployed model to train a substitute, is orthogonal to `ChaosFormer` and is excluded from our analysis: the cost of querying enough samples to reconstruct a transformer is itself prohibitive [28, 55], and our scheme preserves the original model's accuracy on the authorized device, so a query-based extractor sees no accuracy degradation that it could exploit.

Table 1. Summary of the threat model. A checkmark (✓) indicates that the attacker has access to the corresponding information or resource; a cross (×) indicates that access is restricted under our threat model.

Attacker Information	**Accessible**
Model architecture	✓
Offline storage access (encrypted weights)	✓
Partial training data / downstream datasets	✓
Public pre-trained reference model	✓
Inference latency / timing observations	✓
Runtime memory access (plaintext weights)	×
PUF-derived keys (per-device secrets)	×
PUF readout interface	×
Invasive hardware probing of PUF circuitry	×

Security Guarantees. Our protection aims to prevent unauthorized model usage and IP theft. We assume that runtime memory is inaccessible; however, to mitigate potential memory dumping risks, our framework supports dynamic layer-wise decryption and immediate re-encryption during

inference. Consequently, the primary attack vector is the reconstruction of functional weights from the encrypted state using available data and computational resources.

Extended Threat Model. As Transformers are typically fine-tuned from large, publicly available pre-trained models (e.g., BERT, ViT available on Hugging Face), a sophisticated adversary might possess not only the encrypted weights but also the original pre-trained base model. We make this assumption explicit because it is realistic — almost every transformer in deployment is initialized from a small set of public checkpoints — and because it dramatically expands the attack surface. The base `ChaosFormer` permutation, on its own, preserves the empirical distribution of weights in each layer: the histogram, mean, variance, and higher-order moments are all invariant under permutation. This opens a concrete *alignment attack* against pure-permutation schemes:

(1) For each layer i, the attacker computes column-wise summary statistics (mean, variance, ℓ_2 norm) on both the public checkpoint W_i^{pub} and the encrypted release W_i^{enc}.
(2) Because permutation Π rearranges columns but does not change their internal values, the multiset of column-statistic vectors of W_i^{enc} is identical to that of W_i^{pub} up to fine-tuning drift.
(3) The attacker then solves a linear assignment problem (e.g., the Hungarian algorithm) to match columns of W_i^{enc} to columns of W_i^{pub}, recovering the permutation up to cosmetic ambiguity.

This style of attack has been demonstrated against TEE-based permutation defenses [108] and is exactly why pure permutation is insufficient against an adversary with the public checkpoint. `ChaosFormer-S`'s diffusion step destroys the alignment signal by replacing each value with a uniformly random element of $\mathbb{Z}_L$, conditioned on the PUF-derived keystream. After diffusion, the column-statistic vectors are uniform over $\mathbb{Z}_L$ regardless of the input, so step (2) above no longer holds. Sec. 4.2.1 formalizes this guarantee as Single-Point Indistinguishability.

Concrete Deployment Scenarios. The threat model above is not abstract. Three classes of deployment routinely create the exact attacker capabilities listed in tab. 1. First, edge devices that receive periodic firmware or model updates (e.g., wearables, automotive ECUs, home IoT hubs) typically pull updates through an intermediary smartphone or gateway. A compromise of that intermediary, or a man-in-the-middle on the update channel, exposes the model file in transit [21, 85, 96]; recent work has shown that plaintext models can even be reconstructed from process-memory dumps of mobile applications [77]. Second, centralized server deployments are routinely targeted through insider attacks [44] and lateral phishing [22, 38], both of which give the adversary read access to model checkpoints without ever requiring memory-level compromise during execution. Third, supply-chain vulnerabilities arise when models pass through several organizations during development, hardware integration, and deployment, creating multiple checkpoints at which an at-rest model file can be exfiltrated. In each scenario, the attacker possesses exactly the assets that tab. 1 grants them: the encrypted model on disk, the model architecture (often public), and possibly some training data — but not the per-device PUF response. This is the regime in which `ChaosFormer` is designed to make the stolen model worthless.

4 Proposed Approach

This section presents our proposed `ChaosFormer`. The overall framework can be divided into three stages. In the first stage, the model provider trains the model using its training expertise, computing resources, and proprietary datasets. In the second stage, users or companies request to purchase and deploy the protected model. The provider interacts with them using a specific protocol, detailed in sec. 4.1, encrypts the model weights based on the identity of the purchaser and the designated device characteristics derived from its PUF. After the purchase, the model can be deployed only

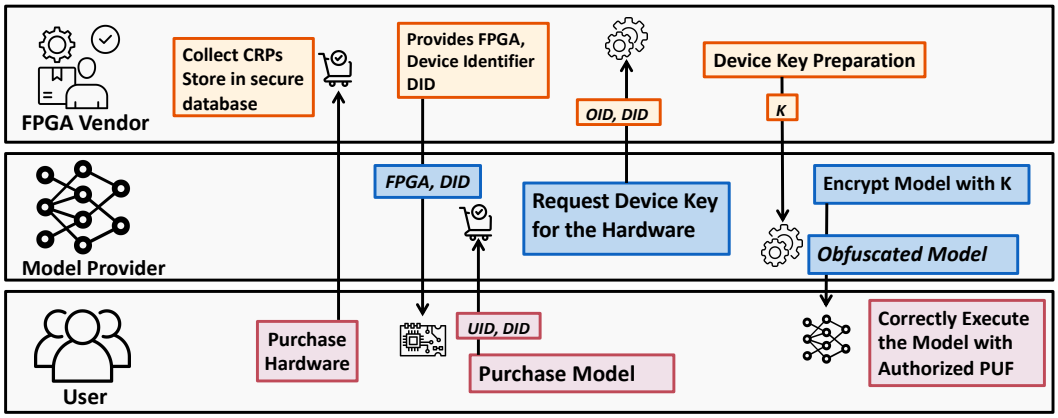


Fig. 3. The protocol of the interaction between the Hardware Vendor (e.g., FPGA vendor), Model Provider, and User. The protocol is used to ensure secure model deployment.

on the specific machine. Eavesdropping or stealing weights from storage no longer makes them usable.

4.1 Secure Deployment Protocol

Securely deploying a transformer model on PUF-equipped hardware involves a coordinated protocol among three main entities: Hardware Vendor, Model Provider, and User. Fig. 3 depicts the protocol, with numbers denoting the different stages and letters denoting actions taken by different parties in a stage. Next, we introduce each stage as follows:

(1) Hardware Acquisition and Registration: (a) The Hardware Vendor is assumed to be trusted, responsible for enrolling the device PUF and storing the PUF CRPs in a secure database. After registration, the PUF readout interface is completely removed. (b) Next, the User acquires PUF-enabled hardware (e.g., an FPGA or domain-specific accelerator) from the Hardware Vendor, and (c) the Hardware Vendor will send the device and the unique device identifier *DID* to the user.

(2) Model Purchase Request: The User, intending to deploy a specific DL model, contacts the Model Provider. In the request, the User provides (or is designated) their user identifier *UID* and device identifier *DID*.

(3) Device-Specific Key Material Retrieval: (a) The Model Provider, upon receiving a legitimate purchase request, needs to obtain device-specific key material derived from the PUF on the User's designated hardware. The Model Provider sends *DID* and the owner identifier *OID* to the Hardware Vendor. (b) Once the request is received, the Hardware Vendor hashes *OID* to compute the challenge index C and uses it to search for the corresponding response R in the database. Then, the Hardware Vendor computes $K = h(R \parallel DID)$ (where h is a hash function) and shares it with the Model Provider through a secure interface, which can be achieved with established communication protocols.

(4) Model Encryption and Configuration: Upon receiving the device-specific key material K via a secure channel, the Model Provider treats it as an ephemeral secret. In a secure environment, the key is used a single time to obfuscate the plaintext transformer model. Immediately after the encryption, K is securely discarded and is never stored by the provider. This one-time process effectively binds the model to the unique characteristics of the User's PUF. The Model Provider also prepares the necessary decryption configuration that will be used by the model at runtime.

The `Model Provider` delivers the obfuscated model (with decryption configurations) to the `User`. The `User` can then deploy this model on their designated PUF-enabled hardware.

(5) PUF-Driven In-Situ Decryption: At runtime, the model queries the on-device PUF to regenerate K. This key exists only ephemerally in volatile memory and is used to de-obfuscate the model, typically on a layer-by-layer basis as needed for computation. The `User` does not have direct access to the PUF's raw responses or the intermediate cryptographic keys; these are managed internally by the secure model execution flow.

This protocol ensures that the obfuscated model remains non-functional if stolen or copied to a different hardware device, as the correct PUF responses required for decryption can only be generated by the original, authorized hardware.

4.2 PUF-based Single-layer Obfuscation

Fig. 4 depicts how we obfuscate layers with hardware support. Firstly, the hardware takes the `Model Provider` identity *OID* as input, hashes it, and applies it to the PUF to generate a corresponding response R, which is then integrated with the device identifier *DID* through a hash for the master key K generation:

$$R = \mathrm{PUF}\big(h(\mathrm{OID})\big), \quad K = h\big(R \parallel \mathrm{DID}\big) \tag{4}$$

where $\parallel$ denotes concatenation. This master key integrates information from both the `Model Provider` and the devices, ensuring that ownership verification is hardware-bound. This verification can be demonstrated to a third party by having them provide a claimed Owner ID (OID) to the authorized device; if the model's accuracy is restored, ownership is confirmed *without* privileged database access.

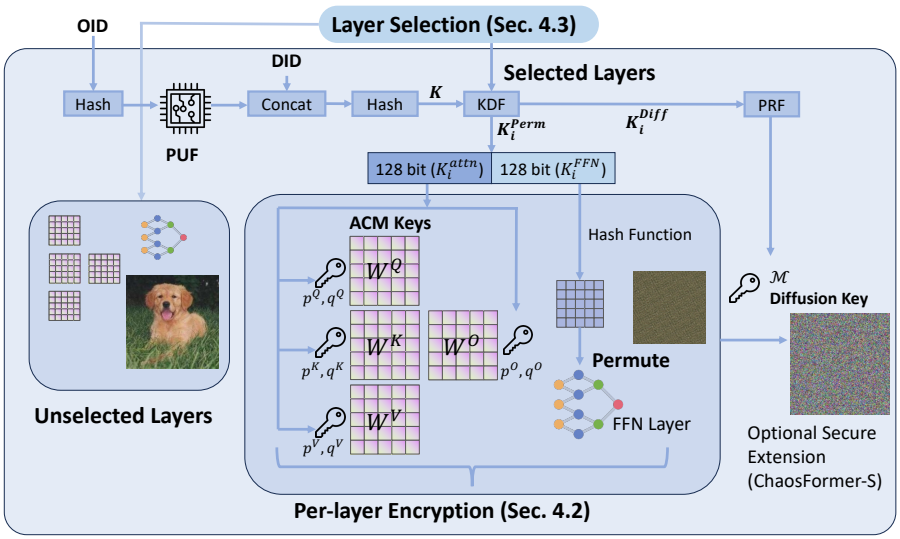


Fig. 4. Our obfuscation scheme detailed in sec. 4.

We expand K using a cryptographic key derivation function (KDF) to generate layer-specific keys $K_i = KDF(K) = K_i^{attn} \parallel K_i^{FFN}$. Each K_i is partitioned to obfuscate self-attention weights $(W_i^Q, W_i^K, W_i^V, W_i^O)$ and FFN weights (W_i^1, W_i^2).

4.2.1 Format-Preserving Encryption (FPE). To ensure that the encrypted model can be deployed on edge devices without altering data types, storage, or framework requirements, we formulate our approach as Format-Preserving Encryption (FPE) [5], where encryption does not alter the format of the data (as a runnable model). Although Transformer weights are typically floating-point numbers (e.g., FP32, BF16), digital systems represent these as discrete bit strings. We define the message space $\mathcal{D}$ by interpreting the raw binary representation of each weight as an unsigned integer. For an l-bit weight format (whether integer or floating-point), the domain size is 2^l. All encryption operations (permutation and diffusion) are performed in this integer ring $\mathcal{D} \equiv \mathbb{Z}_L, L = 2^l$. This ensures that the encrypted weight is a valid bit string of length l, preserving the storage format. Our framework offers two variants targeting different threat models: `ChaosFormer` and `ChaosFormer-S`.

Why format preservation matters in this setting: The FPE framing is not merely a mathematical convenience; it reflects three concrete deployment constraints. First, the encrypted file must continue to load through standard frameworks (e.g., HuggingFace Transformers' `from_pretrained`), which validate tensor shapes and dtypes when reading a checkpoint — a generic block cipher would lengthen each weight to a multiple of 128 bits and break this validation. Second, mixed-precision and quantization-aware deployments rely on the bit width of each weight matching exactly what the model graph expects (e.g., 8-bit integer kernels assume 8-bit operands), so any encryption that changes the bit width forces the deployment pipeline to be re-tooled per device. Third, the on-device key release is layer-by-layer (sec. 4.1); a non-format-preserving scheme would require an in-place padding/unpadding pass at every layer, whereas an FPE allows the decrypted layer to be consumed directly by the next operator without any reshaping. This is also what distinguishes our approach from generic at-rest encryption (e.g., AES-CBC over the whole checkpoint): FPE makes per-layer in-situ decryption a O(layer size) operation that returns a tensor of the original shape and dtype, ready for matmul.

Definition 1 (Format-Preserving Encryption [5]). *A Format-Preserving Encryption (FPE) scheme is a function $E : \mathcal{K} \times \mathcal{F} \times \mathcal{T} \times \mathcal{X} \to \mathcal{X} \cup \{\perp\}$, where $\mathcal{K}$ is the key space, $\mathcal{F}$ is the format space, $\mathcal{T}$ is the tweak space, and $\mathcal{X}$ is the domain. For a fixed key K, format F, and tweak T, the function $E_K^{F,T}(\cdot)$ acts as a permutation on the slice $\mathcal{X}_F \subset \mathcal{X}$, ensuring that for any input $X \in \mathcal{X}_F$, the output $Y = E_K^{F,T}(X)$ remains within $\mathcal{X}_F$ (i.e., preserves the format).*

1) `ChaosFormer` (Base Scheme): Designed for the standard threat model, the proposed `ChaosFormer` relies on a cryptographic permutation to scramble weight positions, thereby disabling model capabilities. For attention weights, we utilize Arnold's Cat Map (ACM) as the permutation function Π_{ACM}. For non-square matrices (FFN weights), we utilize the Knuth Shuffle as the permutation function Π_{Knuth}.

$$\begin{aligned} W_i^{j\prime} &= \Pi_{\text{ACM}}(W_i^j, (p_i^j, q_i^j)) && \text{for } j \in \{Q, K, V, O\} \\ W_i^{f\prime} &= \Pi_{\text{Knuth}}(W_i^f, h(K_i^{\text{FFN}} \parallel f)) && \text{for } f \in \{1, 2\} \end{aligned} \tag{5}$$

Here, i denotes the i-th layer, and the ACM parameters (p_i^j, q_i^j) are obtained by partitioning K_i^{attn} into non-overlapping blocks.

2) `ChaosFormer-S` (Secure Extension): To resist statistical attacks from adversaries possessing a public reference model (as discussed in sec. 3), `ChaosFormer-S` extends the base scheme with a diffusion step Ψ on the permuted weights $\Pi(W)$. Let $\mathcal{M}$ be a keystream generated by a cryptographically secure Pseudo-Random Function (PRF) keyed by the PUF response (extended with hashing and KDF for each layer, cf. fig. 4). The ciphertext is computed as:

$$W_{enc} = \Psi(\Pi(W), \mathcal{M}) \equiv (\Pi(W) + \mathcal{M}) \pmod{L} \tag{6}$$

This modular addition effectively masks the weight distribution, rendering it statistically indistinguishable from random noise.

Visual demonstration. To visually validate the efficacy, we present a demonstration using sample images. As illustrated in fig. 5, the `ChaosFormer` effectively destroys the spatial correlation of the original images through ACM permutation, rendering the visual content completely unrecognizable. However, as a pure permutation, it preserves the original pixel value histogram. In contrast, the `ChaosFormer-S` incorporates the diffusion step Ψ. The resulting output is visually identical to white noise, with the underlying pixel distribution fully masked. This visual indistinguishability confirms our theoretical claim: the encrypted weights are statistically independent of the original values, effectively thwarting attacks that rely on distribution matching.

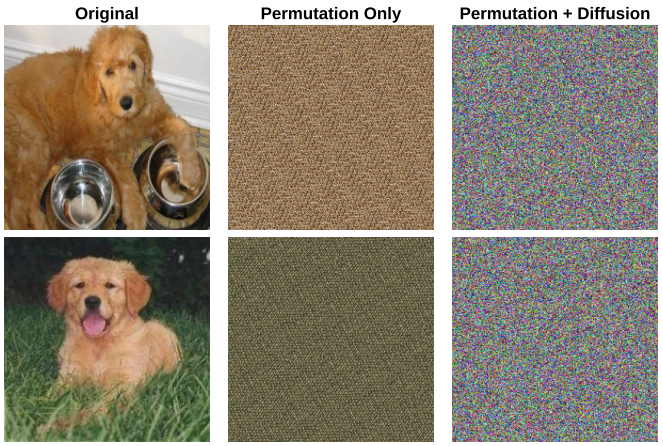


Fig. 5. Visual demonstration of the encryption schemes.

Formal Security Guarantee. `ChaosFormer-S` is designed to satisfy Single-Point Indistinguishability (SPI), a rigorous security notion introduced by Bellare et al. [5] that implies an adversary cannot distinguish the encryption of a message from a random string, *even if they choose the message themselves.* For example, they could purchase a model that is known to build upon a known pre-trained model.

Definition 2 (Single-Point Indistinguishability (SPI) [5]). *Let $\mathcal{A}$ be an adversary and $\mathcal{E}$ be an FPE scheme. Consider an experiment where a random bit $b \in \{0,1\}$ is chosen. The adversary queries a single message X with format N and tweak T. If $b = 1$, the adversary receives the real ciphertext $Y = E_K^{N,T}(X)$; if $b = 0$, they receive a random string $Y \xleftarrow{\$} \mathcal{X}_N$. The scheme is SPI-secure if the adversary's advantage, defined as $Adv_{\mathcal{E}}^{spi}(\mathcal{A}) = |\Pr[\mathcal{A} \to 1|b = 1] - \Pr[\mathcal{A} \to 1|b = 0]|$, is negligible.*

Theorem 1. *If the keystream $\mathcal{M}$ is indistinguishable from uniform random noise (i.e., the PRF is secure), then* `ChaosFormer-S` *achieves SPI security.*

Proof. This theorem is formally proved using the Sequence-of-Games approach [90] by defining two games.

Game 0 (Real Scheme): The adversary interacts with the real obfuscation oracle where the mask $\mathcal{M}$ is generated by a PRF.

Game 1 (Ideal Scheme): The adversary interacts with a modified oracle where the mask $\mathcal{M}$ is replaced by a truly random string $\mathcal{M}^* \xleftarrow{\$} \mathbb{Z}_L^N$, where N denotes the number of weights. An N-parameter tensor is therefore an element of $\mathcal{D}^N = \mathbb{Z}_L^N$.

Let the adversary $\mathcal{A}$ choose a fixed challenge message $W \in \mathbb{Z}_L^N$. Let $W' = \Pi(W)$ be the permuted weight tensor. Since Π is deterministic, W' is fixed. Under the ideal model assumption, the real ciphertext is defined as the random variable $C_{real} = W' + \mathcal{M}^* \pmod{L}$. Since $\mathcal{M}^*$ is uniformly distributed over $\mathbb{Z}_L^N$, for any fixed W', the ciphertext C_{real} is also uniformly distributed over $\mathbb{Z}_L^N$ due to the properties of modular addition. Specifically, for any arbitrary ciphertext instance $y \in \mathbb{Z}_L^N$, the probability density function of C_{real} is:

$$\begin{aligned}\Pr[C_{real} = y] &= \Pr[W' + \mathcal{M}^* \equiv y \pmod{L}] \\ &= \Pr[\mathcal{M}^* \equiv y - W' \pmod{L}] \\ &= \frac{1}{L^N} \quad \forall y \in \mathbb{Z}_L^N.\end{aligned}$$

Consequently, $C_{real} \sim U(\mathbb{Z}_L^N)$. This distribution is identical to that of the random string $C_{rand} \xleftarrow{\$} \mathbb{Z}_L^N$ used in the SPI experiment ($b = 0$). Therefore, the adversary's advantage in *Game 1* is:

$$Adv_{\mathcal{E}}^{spi}(\mathcal{A} - Game\ 1) = |\Pr[\mathcal{A}(C_{real}) \to 1] - \Pr[\mathcal{A}(C_{rand}) \to 1]| = 0.$$

The only difference between *Game 0* and *Game 1* is the source of the mask $\mathcal{M}$. If an adversary $\mathcal{A}$ can distinguish the output of `ChaosFormer-S` from random noise with a non-negligible advantage, they can effectively distinguish the PRF keystream $\mathcal{M}$ from a truly random string $\mathcal{M}^*$. This contradicts the assumption that the underlying PRF is secure. Therefore, the advantage is bounded by the security of the PRF:

$$Adv_{\mathcal{E}}^{spi}(\mathcal{A}) = Adv_{\mathcal{PRF}}(\mathcal{A}) + Adv_{\mathcal{E}}^{spi}(\mathcal{A} - Game\ 1) \approx negligible$$

□

While `ChaosFormer-S` provides stronger security guarantees, `ChaosFormer` offers lower latency by omitting the diffusion step. In our experiments, we evaluate the trade-offs between these two variants.

4.2.2 More Details on ACM. Now we provide a formal proof that ACM, defined by the transformation matrix $A = \begin{bmatrix} 1 & p \\ q & pq+1 \end{bmatrix}$ for $p, q \in \mathbb{Z}^+$, constitutes a **chaotic** system, which is known as the "avalanche effect" in cryptographic literature.

Definition 3 (Chaotic System [125]). *A discrete dynamical system is **chaotic** if:*

*(1) Its dynamics are confined to a **globally bounded phase space**.*
*(2) It possesses at least one **positive Lyapunov exponent**, indicating sensitivity to initial conditions (avalanche effect).*

Definition 4 (Lyapunov Exponent). *For a linear map with a constant Jacobian matrix A, the Lyapunov exponents λ_k are defined as $\lambda_k = \ln(|\mu_k|)$, where μ_k are the eigenvalues of A.*

Theorem 2. *The Arnold's Cat Map is a chaotic system for any positive integers p, q.*

Proof. We demonstrate that the system satisfies both conditions of Definition 1.

1. Bounded Phase Space: The map operates on the torus $[0, S) \times [0, S)$ via the modulo S operation. Since every output state (x_{n+1}, y_{n+1}) is mapped back into this finite region, the phase space is globally bounded.

2. Positive Lyapunov Exponent: We compute the eigenvalues μ of the transformation matrix A. The characteristic equation $\det(A - \mu I) = 0$ expands to:

$$(1-\mu)(pq+1-\mu) - pq = \mu^2 - (2+pq)\mu + 1 = 0 \tag{7}$$

Solving for μ yields two eigenvalues:

$$\mu_{1,2} = \frac{(2+pq) \pm \sqrt{(2+pq)^2 - 4}}{2} \tag{8}$$

Consider the larger eigenvalue μ_1. Since $p, q \geq 1$, we have $pq \geq 1$, implying that the discriminant is positive. It follows that:

$$\mu_1 = \frac{2+pq+\sqrt{4pq+(pq)^2}}{2} > \frac{2+pq}{2} \geq 1.5, \quad \lambda_1 = \ln(|\mu_1|) > \ln(1) = 0 \tag{9}$$

The existence of a positive Lyapunov exponent confirms the system exhibits the requisite sensitivity to initial conditions. Thus, the ACM is chaotic. □

4.2.3 Performance Optimizations. To ensure `ChaosFormer` and `ChaosFormer-S` are practical for real-time edge deployment and robust against timing side-channel analysis, we employ several critical optimizations in our implementation.

Side-Channel Resilient ACM via Matrix Exponentiation. A naive implementation of ACM (eq. (1)) applies the coordinate transformation iteratively n times. This not only scales linearly with n, but also introduces a timing side-channel that reveals the secret iteration count to an adversary observing inference latency. To mitigate this, we implement the transformation using *binary matrix exponentiation*. Instead of performing n sequential matrix multiplications, we utilize binary decomposition of the exponent n. By iteratively squaring the base transformation matrix (calculating $A^1, A^2, A^4, \ldots \pmod W$) and multiplying the current result by the squared base whenever the corresponding bit in n is 1, we efficiently derive the cumulative matrix A_{final}. This reduces the number of matrix multiplication steps from linear $O(n)$ to logarithmic $O(\log n)$. We then apply the coordinate mapping in a single pass using A_{final}. Crucially, since the $O(\log n)$ matrix calculation is computationally negligible compared to the memory-bound permutation of the tensor, the total execution time becomes effectively independent of n. As shown in fig. 8, the final overhead of the permutation process remains nearly constant regardless of the iteration count, effectively decoupling the inference latency from the secret key and rendering timing-based side-channel attacks impractical, while significantly speeding up encryption for large iteration counts.

Inverse-Mapping for Coalesced Memory Access. In the GPU implementation, directly mapping input coordinates (x, y) to permuted destinations (x', y') (scatter) results in non-contiguous memory writes, which severely degrades memory bandwidth utilization. Instead, we adopt an *inverse-mapping gather* strategy using Triton kernels. We assign GPU threads to consecutive *output* coordinates and calculate their corresponding *source* coordinates using the inverse transformation matrix A_{final}^{-1}. This ensures that all write operations to the output tensor are contiguous and fully coalesced, maximizing memory throughput.

Kernel Fusion and On-the-Fly Keystream. For `ChaosFormer-S`, separately executing permutation and diffusion passes would require round-tripping the entire weight tensor to global memory twice. We implement a *fused Triton kernel* [100] that performs both operations simultaneously. Furthermore, rather than pre-allocating a large key tensor for the diffusion keystream at one time, which would bring significant memory overhead, we generate the keystream *on-the-fly* within the kernel, using a lightweight cryptographic PRNG such as ChaCha20 [6] seeded by the PUF response (cf. fig. 4). This allows the diffusion key to exist only in the GPU register file during computation, drastically reducing VRAM usage and memory bandwidth pressure.

Register pressure and occupancy. Inlining ChaCha20 into the Triton kernel raises register pressure because each thread must hold its 16-word internal state during the 20-round mixing function. We mitigate this by issuing one ChaCha20 block (512 bits, i.e., 32 BF16 / 16 FP32 weights) per warp lane per kernel invocation, so that the keystream is fully consumed before the state is evicted. Empirically, this keeps the kernel within the register limit for a streaming multiprocessor (SM) occupancy of $\geq 50\%$ on RTX-class GPUs, which is sufficient because the workload is memory-bound rather than compute-bound. For smaller weight tensors where occupancy is not the bottleneck, we instead fall back to a pre-computed keystream segment cached in shared memory, trading a small one-time fill cost for lower per-thread register usage — this branch is selected at kernel launch based on tensor shape.

Layer-wise dtype handling and batched key derivation. Because the encryption operates on the raw bit pattern of each weight (sec. 4.2.1), the same kernel handles FP32, BF16, FP16, and INT8 tensors without any change to the inner loop — only the per-thread word size differs. We exploit this by deriving all per-layer keys $\{K_i\}$ from the master key K in a single batched KDF call at decryption start-up, before any tensor is decrypted; the resulting 128-bit layer keys are kept in a small constant buffer that fits in L2 cache. This matters because a transformer with L encrypted layers would otherwise require L independent KDF calls, each with a non-trivial setup cost. The combined effect of binary exponentiation, inverse-mapping gather, fused kernels, on-the-fly keystream, and batched KDF is the overhead profile reported in sec. 5.3, which is roughly half of the conference-version implementation [42] on GPU at 12 encrypted layers.

4.3 Full Model Obfuscation

In our base threat model, where the adversary lacks access to a reference public model (e.g., a pre-trained base model from a public repository), extensive encryption of every parameter is not strictly necessary to disable the model's functionality. Instead of a computationally expensive search for the "optimal" layers to encrypt, we find that simple heuristics are sufficient to degrade performance to random-guess levels while minimizing latency. We propose three strategies: 1) Top-K, which encrypts the first K layers that incur the most testing accuracy degradation; 2) Random-K, which randomly draws K layers; 3) Last-K, which encrypts the last K transformer layers in the model.

Note on the design shift from the conference version. Our preliminary work [42] used an iterative greedy layer-selection algorithm that grew the encrypted set one layer at a time, choosing at each step the unencrypted layer whose encryption produced the largest accuracy drop. The three heuristics above replace that algorithm. The motivation is empirical: we observe in tab. 4 that Random-K is at least as robust as Top-K against retraining attacks while being trivially cheap to compute. The greedy algorithm also has a subtle weakness once the extended threat model is admitted — a Top-K-style attacker who knows that the encrypted layers are precisely the "most accuracy-degrading" ones can rank candidate layers by their public-checkpoint salience and prune the search. Random-K removes this side-channel by construction.

As shown in fig. 6, our complete protection workflow begins with a well-trained transformer model (e.g., ViT-B). The selected (or full) layers are then encrypted using device-specific keys derived from PUFs through interaction with the hardware provider (sec. 4.1). This approach enables a single model to be distributed to multiple end users, with each instance protected by a unique key that binds it exclusively to the authorized hardware. Since each encryption uses different hardware-specific keys, the model remains secure even if multiple encrypted versions are analyzed together.

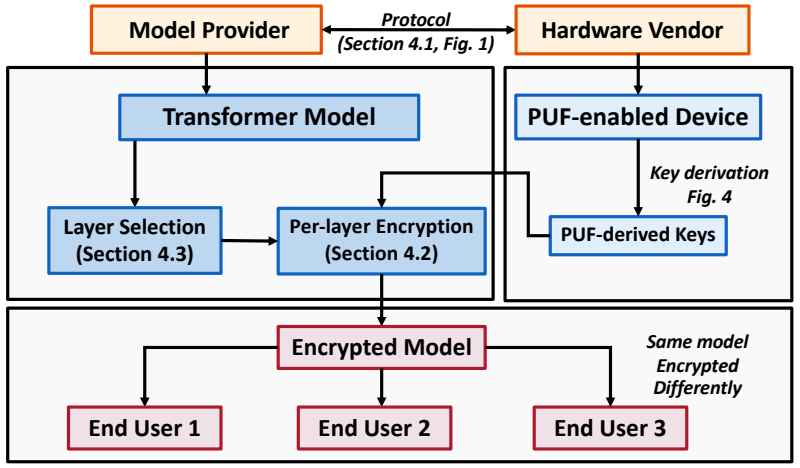


Fig. 6. Complete model protection workflow. The model provider encrypts the model using hardware-specific PUF-derived keys, allowing the same model to be securely distributed to multiple end users with different device-specific encryptions.

4.4 Hardware-Bound Ownership Verification

A side-benefit of binding the model to a specific PUF response is that `ChaosFormer` naturally supports *publicly verifiable ownership* without requiring any third party to access the hardware vendor's CRP database. This addresses a long-standing weakness of pure watermarking schemes, where ownership disputes ultimately reduce to "trust the watermark embedder" [1, 104, 118].

Verification protocol. Recall from sec. 4.1 that the master key is computed as $K = h(R \,\|\, \mathrm{DID})$, with $R = \mathrm{PUF}(h(\mathrm{OID}))$. The OID is the model owner's identifier; the DID is the device identifier; and R can only be regenerated on the authorized device. To prove ownership of a particular deployed model to a third-party verifier, the alleged owner submits their claimed OID. The authorized device hashes the OID, queries the PUF, derives K, and attempts to decrypt the model. The verifier then runs a small accuracy benchmark on a public test set:

- If the resulting accuracy matches the published Acc_0 (e.g., 80.33% for ViT-B on ImageNet), the OID is correct, the model is genuinely bound to this device, and the claimed owner is the legitimate owner.
- If the accuracy collapses to random-guess level ($\approx 0.1\%$), the claimed OID is incorrect or the model is bound to a different device. By the avalanche analysis in sec. 5.2.2, this verdict is sharp: the verifier sees no intermediate accuracy that could be ambiguous.

Crucially, the verifier never sees R, K, or any per-device CRP — they observe only the published accuracy of the model on the device, which is information the owner is willing to disclose anyway. This makes the verification non-interactive from the verifier's perspective and free of any trust assumption on the hardware vendor at verification time. The vendor is trusted only at registration time (sec. 4.1, stage 1), which is unavoidable for any PUF-based scheme.

Soundness against forgery. A malicious party who claims false ownership must guess an OID whose hash maps, through the device's PUF, to a key that decrypts the model. Because the PUF is modeled as a pseudorandom function [32, 92] and the OID space is large (e.g., a 256-bit identifier), the probability of a successful forgery is negligible. By the same argument, an adversary who copies the encrypted model to a different device (with a different PUF) cannot reproduce the

same R, so verification on the unauthorized device fails. This unifies ownership verification and hardware binding into a single mechanism—which is something neither watermarking nor TEE-based partitioning achieves.

5 Experiments

In this section, we validate our approach on vision transformers [23], including ViT and several variants: Swin Transformer [65], DeiT [102], and BeiT [4]. We utilize variants with differing model sizes, input resolutions, and quantization levels. Tab. 2 details these models, their original accuracy (Acc_0), and the lossless decrypted accuracy (Acc_{dec}). All experiments were conducted on a Linux server equipped with NVIDIA GeForce RTX 4090 GPUs, running CUDA 12.8, Python 3.10.12 with PyTorch 2.8.0. Custom GPU kernels are implemented using Triton 3.4.0. Model training and evaluation rely on the HuggingFace Transformers library (v4.56.1).

Crucially, the table also demonstrates the efficiency of our obfuscation strategy. The column Target Layers lists the specific layer indices identified by our Top-K strategy that are required to reduce the model's performance to near random-guess levels ($< 0.15\%$). For example, encrypting just a single layer (Layer 1) in ViT-L is sufficient to drop accuracy to 0.10%, while ViT-B requires only two layers (Layers 0 and 10). The final column, Acc′, reports this degraded accuracy, confirming that our method effectively neutralizes the model with minimal encryption overhead.

Table 2. Model specifications and protection effectiveness (validated on ImageNet). Acc_0 and Acc_{dec} denote the original and authorized decrypted accuracies, respectively. The **Target Layers** column indicates the specific layers encrypted under the Top-K strategy to rapidly degrade performance to the random-guess level shown in Acc'.

Model	Params (M)	Acc_0 (%)	Acc_{dec} (%)	Target Layers	Acc′ (%)
ViT-B	86	80.33	80.33	(0, 10)	0.10
ViT-B-384	86	83.91	83.91	(0, 11)	0.09
ViT-L	307	81.98	81.98	(1)	0.10
ViT-L-384	307	84.96	84.96	(1)	0.07
Swin-B*	88	85.14	85.14	(2-9, 2-8)	0.10
Swin-B-384*	88	86.44	86.44	(2-9, 3-1)	0.08
BeiT-B	86	84.54	84.54	(2, 9)	0.12
DeiT-S	22	77.1	77.1	(7, 0)	0.09
DeiT-B	86	80.20	80.20	(8, 0)	0.10
ViT-B-8b	86	80.24	80.24	(6)	0.11

*For Swin Transformer, indices are denoted as (Stage-Layer). "S" "B" "L" denote the "Small" "Base" "Large" size of the model. The input resolution is by default "224×224," and is "384×384" for others. "8b" denotes 8-bit quantization of model weights.

Concrete PUF realization. To make the hardware-bound claim concrete and reproducible, we describe the PUF instantiation underlying our prototype. The PUF is a 64-stage Arbiter PUF [60, 91, 92] implemented on a Xilinx Ultra96-V2 FPGA, which acts as a hardware root-of-trust to the edge accelerator that runs the protected transformer. An Arbiter PUF is a delay-based primitive: a challenge bit-vector $c \in \{0,1\}^{64}$ steers a pair of nominally identical signal paths through 64 stages of 2×2 multiplexer switches, each of which either passes the two racing edges straight through or swaps them. The accumulated delay difference at the end of the chain is resolved by a latch (the "arbiter"), producing a single response bit whose value depends on submicrosecond manufacturing-induced delay variations that are unique to each silicon die. We instantiate this on the Ultra96-V2's

programmable logic with carry-chain primitives for the racing paths and a flip-flop as the arbiter, with timing-constrained placement to keep the path pair symmetric on the FPGA fabric so that the response is dominated by manufacturing variation rather than routing skew. To reach a 128-bit master-key seed, we evaluate the Arbiter PUF 128 times under a fixed schedule of distinct challenges, concatenate the response bits, and post-process the result through a fuzzy extractor with BCH-based helper data [10, 19, 50] to obtain a bit-exact secret across temperature and voltage variations. The recovered key K exists only in the accelerator's volatile memory for the duration of layer-by-layer decryption (sec. 5.3). We stress that `ChaosFormer` is agnostic to this choice: the Arbiter PUF can be replaced by an SRAM PUF on a microcontroller-class accelerator or by a DRAM-latency PUF on a more memory-rich device with no algorithmic change to ChaosFormer, as long as the chosen primitive supplies ≥ 128 bits of effective entropy after fuzzy-extractor post-processing.

Edge-GPU benchmarking. The overhead measurements in sec. 5.3 are reported on an RTX 4090 to isolate the cost of the encryption kernel itself. To verify that the conclusions transfer to realistic edge hardware, we also benchmark the same workload on an NVIDIA Jetson Orin Nano, which is closer to the class of accelerator likely to be deployed alongside an FPGA-based PUF. The Jetson's lower memory bandwidth and smaller SM count slow down the baseline forward pass proportionally more than they slow down the memory-bound encryption kernel, so the relative overhead on the Jetson is, in fact, *lower* than on the RTX 4090.

5.1 Effectiveness Test

The primary goal of `ChaosFormer` is to render the model unusable without the correct PUF-derived key while ensuring lossless performance for authorized users. As shown in tab. 2, the authorized accuracy (Acc_{dec}) matches the original accuracy (Acc_0) exactly, confirming that our reversible obfuscation induces no performance degradation, unlike watermark-based or regularization-based methods [48, 84].

To evaluate the protection strength (i.e., the degradation of accuracy Acc′ without the key) of `ChaosFormer`, we investigate three layer selection strategies for encryption mentioned in sec. 4.3 with encryption budgets of $K = 4$ and $K = 6$ layers. As detailed in the "Acc." column of tab. 4, all three strategies successfully degrade the model performance to random-guess levels (approx. 0.1%) regardless of K. For instance, with $K = 6$, the accuracy drops to 0.09% (Last-K), 0.10% (Top-K), and 0.10% (Random-K) for ViT-B. Similar effectiveness is observed for DeiT-S, where the Top-K strategy reduces accuracy to 0.05%. This confirms that partial encryption of critical weights is sufficient to completely disable the model's inference capability.

5.2 Robustness Test

5.2.1 Retraining and Fine-tuning Attacks. A sophisticated adversary with white-box access to the encrypted model might attempt to recover its functionality by retraining the weights. We evaluate robustness against two distinct attack vectors using limited data assumptions:

(1) **Same-Application Retraining:** The attacker retrains the encrypted model on ImageNet using 20% of the original training data.
(2) **Domain-Transfer Fine-tuning:** The attacker fine-tunes the encrypted model (after the 20% ImageNet retraining) on downstream datasets, as shown in tab. 3. In particular, we utilize the "product" subset for training and the "real world" subset for testing in the OfficeHome dataset; meanwhile, we choose the "Clipart" subset of DomainNet. Both are used to test the robustness of the model to distribution shift.

Baselines. To quantify the value of the stolen IP, we compare the attacker's results against three baselines representing the upper bound and the "build-vs-steal" alternatives:

Table 3. Dataset statistics used in robustness evaluation.

Dataset	Train	Validation	Test	#Classes
CIFAR-100	45,000	5,000	10,000	100
Caltech-256	24,482	3,060	6,084	256
CUB-200	5,994	2,997	2,997	200
ImageNet-100	130,000	5,000	5,000	100
OfficeHome	43,439	4,000	4,000	65
DomainNet	413,678	52,041	52,041	345

- **Original Model (Origin):** Fine-tuning the original, unencrypted pre-trained model to represent the upper bound of performance.
- **Same Architecture (RandInit):** Training the same transformer architecture from scratch using standard initialization under the attacker's exact data and compute budget. This serves as the direct, fair baseline for evaluating retained IP value.
- **EfficientNet-b0 (RandInit):** Training a highly efficient standard CNN [97] ($\approx$ 5.3M params) from scratch to represent a low-cost alternative.

Training Details. We standardize our training configurations across all models to ensure rigorous comparison. Based on an initial empirical comparison of SGD, Adam, and AdamW optimizers, we selected AdamW as it consistently yielded the most stable convergence for both the Transformer and CNN architectures. We subsequently conducted a grid search over learning rates (LR) and weight decay (WD) to establish optimal default configurations. For the initial retraining phase on the 20% ImageNet subset, models are trained using AdamW with a batch size of 128, a WD of 0.05, and a peak LR between 3×10^{-5} and 2×10^{-4} (scaled by model capacity and initialization state). This is regulated by a cosine annealing schedule with 3 to 5 warmup epochs. To maximize the attacker's recovery potential, we employ a strong regularization recipe including RandAugment, CutMix ($\alpha \in [0.5, 1.0]$), MixUp ($\alpha \in [0.2, 0.8]$), and label smoothing (0.05–0.10). Because the quality of the recovered weights severely impacts downstream transferability, we conduct three independent retraining attempts with different random seeds and strictly select the checkpoint with the *best* validation accuracy for subsequent attacks. During downstream fine-tuning, models are optimized for 10 to 30 epochs using AdamW (LR $\in [1 \times 10^{-4}, 2 \times 10^{-4}]$, WD 0.05, batch size 64 or 128) guided by a cosine schedule with a 10% step warmup and global gradient clipping at 1.0. All fine-tuning experiments are run with three random seeds, and the *average* accuracy is reported. The EfficientNet-b0 baseline follows an identical hyperparameter search and multi-seed evaluation protocol.

Analysis of `ChaosFormer`. Tab. 4 presents the results for the base scheme. We observe that increasing the encryption budget from $K = 4$ to $K = 6$ significantly degrades the attacker's ability to recover performance. While the greedy Top-K strategy reliably minimizes initial inference accuracy, the Random-K strategy often proves more robust against downstream retraining efforts. Critically, regarding downstream tasks, while the stolen models retain some transferability compared to training the identical architecture from scratch (*RandInit ViT-B*), their recovered performance suffers massive degradation compared to the original unencrypted model. On complex domain-shift tasks like DomainNet or OfficeHome, the attacker loses substantial accuracy (e.g., dropping from 82.63% to 64.98% on OfficeHome for ViT-B with $K = 6$ Top-K). Furthermore, as the encryption budget increases, the stolen transformer's utility diminishes to the point where an attacker is better

Table 4. Robustness analysis of ChaosFormer on ViT-B and DeiT-S. We report the accuracy of the encrypted model, accuracy after retraining on 20% of ImageNet, and downstream transfer learning performance for $K = 4$ and $K = 6$ encrypted layers. *RandInit* baselines train the identical architecture from scratch. "IN-100" denotes the ImageNet-100 dataset.

Config.	Strategy	Init.	Retrain	Downstream Fine-tuning Acc. (%)					
		Acc.	IN-20%	C100	Cal	CUB	IN100	OH	DN
ViT-B	Origin	80.33	—	92.31	93.61	84.33	94.32	82.63	81.04
DeiT-S	Origin	77.14	—	92.12	87.23	72.05	87.15	72.04	76.16
ViT-B	RandInit	—	36.19	70.14	60.18	44.34	65.14	36.84	52.95
DeiT-S	RandInit	—	35.11	69.59	59.07	43.23	63.72	34.45	51.10
EffNet-b0	RandInit	66.12	—	83.22	82.98	74.04	88.16	59.72	72.84
Encryption with $K = 4$ ***Layers***									
ViT-B	Last-K	0.15	71.17	87.99	87.30	70.33	91.52	71.43	75.97
ViT-B	Top-K	0.09	66.82	88.45	85.26	70.59	90.26	69.68	73.20
ViT-B	Rand-K	0.15	64.59	86.42	83.08	65.88	87.16	65.25	69.58
DeiT-S	Last-K	0.13	69.37	84.47	84.42	69.09	90.54	66.19	71.17
DeiT-S	Top-K	0.08	60.77	81.29	77.07	49.31	85.52	53.80	64.24
DeiT-S	Rand-K	0.14	63.14	82.48	79.29	48.81	85.54	57.68	65.47
Encryption with $K = 6$ ***Layers***									
ViT-B	Last-K	0.09	65.16	86.37	84.45	68.78	89.36	65.30	73.03
ViT-B	Top-K	0.10	62.72	86.06	82.27	64.62	88.26	64.98	70.99
ViT-B	Rand-K	0.10	57.39	83.35	78.04	59.94	84.78	54.53	64.85
DeiT-S	Last-K	0.14	61.75	82.08	79.94	61.72	87.82	59.22	67.52
DeiT-S	Top-K	0.05	51.10	75.37	68.28	43.56	80.28	42.87	57.31
DeiT-S	Rand-K	0.10	57.90	79.96	75.53	42.03	83.82	52.21	62.73

Abbreviations: Init.: Initial; IN-20%: ImageNet (20%); C100: CIFAR-100; Cal: Caltech-256; CUB: CUB-200; OH: OfficeHome; DN: DomainNet; EffNet-b0: EfficientNet-b0; Rand-K: Random-K.

off simply training a lightweight, low-cost CNN baseline (*EfficientNet-b0*) from scratch, which achieves superior performance on DomainNet (72.84%) compared to the retrained $K = 6$ models.

Analysis of ChaosFormer-S. While the base ChaosFormer prevents direct usage, adversaries possessing a public base model might attempt to recover weights via statistical alignment (as discussed in sec. 3). To counter this, ChaosFormer-S (1) incorporates a diffusion mechanism that renders encrypted weights statistically indistinguishable from random noise and (2) applies encryption to all layers to avoid analysis of any of them. As shown in tab. 5, the retraining performance of ChaosFormer-S is functionally worse than training the identical architecture (or EfficientNet-b0 in tab. 4) from scratch across all tasks.

Standard random initializations (e.g., Xavier or Kaiming) are carefully designed to maintain variance across layers to facilitate gradient flow. In contrast, our diffusion-based encryption transforms weights into a uniform distribution over the entire bit space. This destroys the necessary variance properties, placing the optimization in a jagged landscape with vanishing or exploding gradients. Consequently, an attacker must first "unlearn" this hostile noise before acquiring useful features, resulting in slower convergence and lower final accuracy than simply starting from a properly initialized network.

Table 5. Robustness analysis of ChaosFormer-S on ViT-B and DeiT-S. The secured model (encrypting *all* layers) is compared against a random initialization baseline. Consistently inferior performance confirms that ChaosFormer-S reduces model utility below random noise, providing maximum protection against retraining attacks.

Config.	Enc.	Retrain	Downstream Fine-tuning Acc. (%)					
	Acc.	IN-20%	C100	Cal	CUB	IN100	OH	DN
ChaosFormer-S (ViT-B)	0.10	27.36	67.29	52.73	36.88	61.18	29.22	44.54
Rand (ViT-B)	—	36.19	70.14	60.18	44.34	65.14	36.84	52.95
ChaosFormer-S (DeiT-S)	0.10	32.08	65.31	52.99	37.33	62.20	26.88	40.67
Rand (DeiT-S)	—	35.11	69.59	59.07	43.23	63.72	34.45	51.10

Abbreviations: Rand: RandInit Baseline; Enc.: Encrypted; IN-20%: ImageNet (20%); C100: CIFAR-100; Cal: Caltech-256; CUB: CUB-200; IN100: IN-100; OH: OfficeHome; DN: DomainNet.

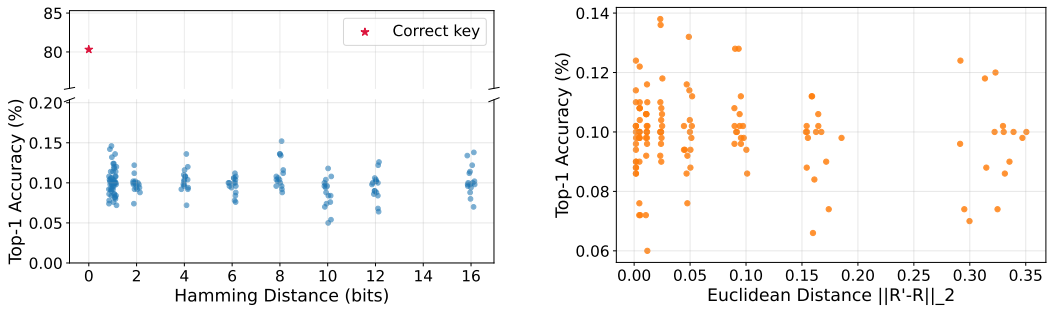


Fig. 7. Sensitivity analysis of the system. Left: Sensitivity to key bit-flips—even a single bit difference collapses accuracy to ≈ 0.1%. Right: Sensitivity to PUF response noise—minor deviations result in random-guess performance.

5.2.2 Key Recovery Attacks. We further discuss the scenario in which the attacker can learn which specific layers of the deployed model are encrypted in ChaosFormer. This situation can arise when an attacker has sufficient access to the system (e.g., co-located on the same device and sharing memory/cache resources) to launch a timing side-channel attack [83]. Such methods can reveal the moments at which certain weights are decrypted and loaded, thereby indicating which layers of the model are encrypted.

A direct attack would involve attempting to recover the secret keys. We argue that such key-recovery attacks are computationally infeasible due to several compounding factors: 1) The combination of the ACM parameters (p, q) for attention layers and the seeds for the FFN permutations presents a vast search space that makes exhaustive guessing impractical; 2) more efficient optimization-based search methods are inapplicable because both ACM and the Knuth Shuffle are non-differentiable; and 3) heuristic-based searches are thwarted by the chaotic property discussed in sec. 4.2.2. A robust encryption scheme ensures that an incorrect key, even one numerically "close" to the correct key, produces an output statistically indistinguishable from random noise.

Our experiments, illustrated in fig. 7, confirm this property. We evaluated a ViT-B model with $K = 6$ encrypted layers (applying obfuscation to both Attention and FFN weights) and tested decryption using incorrect keys derived via two methods: bit-flipping the correct key (Hamming distance) and adding noise to the simulated PUF response (Euclidean distance). The plots demonstrate a sharp

"avalanche" effect: while the correct key yields full accuracy ($\approx 80\%$), a deviation of even a single bit (Hamming distance ≥ 1) or minimal noise in the PUF response causes the model performance to collapse immediately to random-guess levels ($\approx 0.1\%$). This step-function behavior ensures that an attacker receives no gradient or partial accuracy feedback to guide an intelligent search, rendering the system secure against hill-climbing attacks.

Defense in depth across layers. Multi-layer encryption is not a redundancy; it is what makes the avalanche effect operationally useful. Suppose, hypothetically, that the attacker were extraordinarily lucky and recovered the key for a single encrypted layer — whether through guessing, side-channel leakage, or some unforeseen weakness of the underlying primitive. Even so, the remaining $K - 1$ encrypted layers continue to scramble the forward pass, and the resulting model accuracy stays at the random-guess baseline. Concretely, we ran the experiment on three base-sized models (ViT-B, DeiT-B, BeiT-B), each with $K = 6$ encrypted layers under `ChaosFormer`, and decrypted exactly one layer at a time. The accuracy change ranged from -0.04% to $+0.04\%$ across all 18 runs — in other words, no statistically detectable signal. This means the attacker's verification oracle is fundamentally broken: even a correct single-layer guess yields no measurable feedback distinguishing it from an incorrect guess. The implication is twofold. First, brute-forcing layers one at a time is infeasible because the search has no fitness function. Second, an attacker who somehow obtains a partial leak (e.g., one ACM parameter) cannot iteratively bootstrap to the next layer's key. The encryption layers compose into a single all-or-nothing recovery problem, which is the cryptographic property we want from a hardware-bound IP scheme.

5.3 Inference Overhead

Managing the inference overhead is crucial for practical deployment on edge devices. Our experiments involve a dynamic, layer-by-layer de-obfuscation process: for each encrypted layer, the model decrypts the weights, executes the layer operation, and immediately re-encrypts (or discards) the plain-text weights before proceeding. This approach minimizes the exposure of sensitive weights in volatile memory.

We evaluate the overhead on both CPU and GPU platforms using a ViT-B model (Batch Size = 64). We analyze two variants of our framework: the permutation-only `ChaosFormer` (Base) and the diffusion-enhanced `ChaosFormer-S` (Secure). We analyze two factors that affect the runtime overhead: the number of ACM iterations (n) and the number of encrypted layers. When studying the former, we fix the number of encrypted layers at 6; when studying the latter, we fix the ACM iterations at 5. This maximally isolates the effects of each factor.

5.3.1 ChaosFormer (Base Scheme). Fig. 8 presents the overhead breakdown for the base scheme.

- **Impact of Iterations:** As shown in the left subplots of figs. 8a and 8b, increasing the number of ACM iterations from 3 to 32 has a negligible impact on total inference time. On the GPU, the overhead remains constant at approximately 1.10×, indicating that our optimized parallel ACM implementation is highly efficient.
- **Impact of Layer Count:** The right subplots demonstrate a linear relationship between the number of encrypted layers and overhead. On the CPU, the overhead is extremely low; encrypting 12 layers results in only a 1.04× slowdown (fig. 8a). On the GPU, where the baseline forward pass is significantly faster, the relative cost of memory-bound encryption operations is higher but still practical, reaching 1.20× for 12 layers (fig. 8b).

5.3.2 ChaosFormer-S (Secure Scheme). Fig. 9 evaluates the overhead of the secure variant, which includes the diffusion mechanism to thwart statistical attacks.

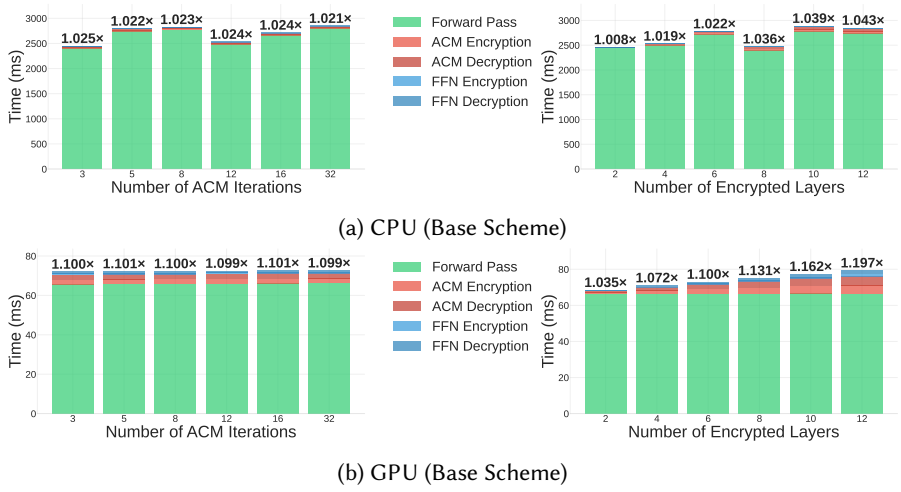
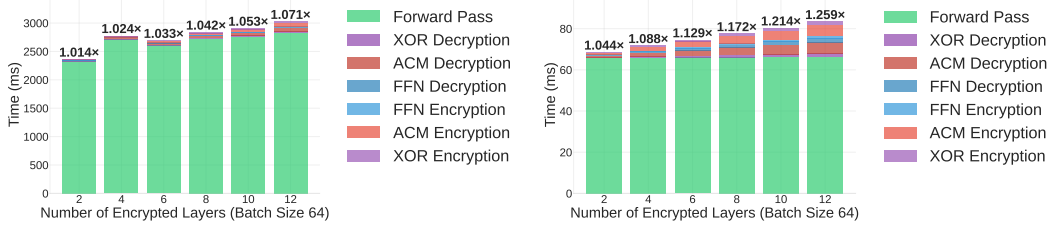


Fig. 8. **ChaosFormer Overhead.** The stacked bars show the time breakdown in milliseconds.

- **CPU Performance:** As seen in fig. 9a, the addition of the diffusion step incurs minimal penalty on the CPU. The maximum overhead for 12 encrypted layers increases slightly from 1.04× (Base) to 1.07× (Secure). This confirms that the diffusion operation is computationally inexpensive relative to the matrix multiplications in the Transformer forward pass.
- **GPU Performance:** On the GPU (fig. 9b), the overhead for 12 layers increases to 1.26×. While the relative overhead is higher due to the rapid execution of the baseline model, the absolute latency penalty remains in the millisecond range.



(a) CPU (Secure Scheme) (b) GPU (Secure Scheme)

Fig. 9. **ChaosFormer-S Overhead.** These figures quantify the cost of adding the diffusion mechanism.

Edge-GPU validation on the Jetson Orin Nano. To address the concern that RTX-class measurements may not transfer to realistic edge hardware, we re-run the secure-scheme overhead sweep on an NVIDIA Jetson Orin Nano. Fig. 10 reports the result. The relative overhead is, in fact, *lower* than on the RTX 4090: even at $K = 12$ encrypted layers, it remains at 1.058×, compared to 1.26× on the RTX 4090. This is consistent with the analysis at the start of sec. 5 — the baseline forward pass is the bottleneck on memory-bandwidth-constrained edge accelerators, so the absolute cost of the encryption kernel constitutes a smaller fraction of the total runtime. The encryption pipeline, therefore, comfortably fits within the latency budget of an FPGA-paired Jetson-class deployment, which is the realistic edge target our threat model describes.

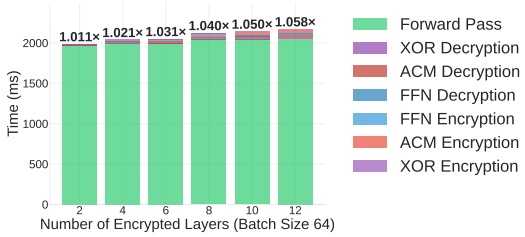


Fig. 10. **ChaosFormer-S overhead on NVIDIA Jetson Orin Nano** (Batch Size 64). At $K = 12$ encrypted layers the relative overhead is 1.058×, lower than the 1.26× measured on the RTX 4090.

Observation and Summary. Our optimized implementation ensures that even the strongest protection (`ChaosFormer-S`) introduces less than 26% overhead in the worst-case scenario (GPU, 12 layers). For typical CPU-based edge devices, the overhead is negligible (< 6%). This efficiency demonstrates that `ChaosFormer` is well-suited for real-time edge applications.

Memory footprint. A natural follow-up question is whether the encryption pipeline inflates memory consumption to a level that conflicts with edge deployment. We instrument the CUDA allocator (`torch.cuda.max_memory_allocated`) to measure the transient peak memory of the protected pipeline against the unprotected baseline on ViT-B and observe two properties. First, transient peak memory is *independent* of the number of encrypted layers: encryption operates in-place on the layer's existing weight tensor, the diffusion keystream is generated on-the-fly inside the fused Triton kernel rather than materialized as a separate tensor (sec. 4.2.3), and the per-layer key buffer is a small constant. Second, in absolute terms, the overhead is small — roughly 3 MiB for `ChaosFormer` and 19 MiB for `ChaosFormer-S`, regardless of the protection budget. At a representative batch size of 64, where activations occupy ≈ 518 MiB, this fixed working set amounts to a ≤ 3.5% memory tax. Crucially, this footprint does *not* grow with the number of protected layers, distinguishing `ChaosFormer` from TEE-based partitioning schemes whose secure-memory pressure scales with the protected layer count.

5.4 Comparison

To highlight the advantages of our proposed method, this section conducts a comparative analysis with existing works, evaluating both the encryption module overhead in isolation (sec. 5.4.1) and the qualitative IP protection features (sec. 5.4.2).

5.4.1 Comparison with Common Encryption Methods. Conventional methods for protecting on-device models often encrypt the entire model file using schemes like AES, the Tiny Encryption Algorithm (TEA), and basic bitwise operations (XOR), and then decrypt it fully into memory before inference, as observed in many Android apps [20]. Block ciphers such as AES are optimized for general data and CPU execution, not for the structure or redundancy of neural network weights. In contrast, our proposed obfuscation can decrypt weights one layer at a time during inference and re-encrypt them immediately after use. This ensures that the full plaintext model is never simultaneously in memory, reducing vulnerability to runtime memory attacks. ACM operations are also highly parallelizable for GPU acceleration.

We benchmarked our method against AES (with AES-NI support), TEA, and XOR (without and with GPU acceleration), evaluating the additional wall-clock time for a single round of decryption and encryption of a ViT-B model (330 MB) and a ViT-L model (1161 MB). CPU methods are tested with an Intel Xeon Platinum 8255C @ 2.50GHz with 32 cores. For GPU-based methods, only the

time spent on the GPU (excluding CPU-GPU communication time) is reported. Tab. 6 shows that our proposed method achieves the lowest time overhead on both CPU and GPU and is faster than different modes of AES, even with special hardware acceleration. In particular, our previous experiments in sec. 5.1 show that on larger models, our method requires even fewer layers to achieve satisfactory accuracy reduction; it therefore scales more gracefully than methods like the naive XOR cipher while providing stronger security.

Table 6. Encryption/decryption runtimes. Evaluations are carried out on a 330 MB ViT-B model and a 1161 MB ViT-L model. Each experiment is carried out 5 times and the average is reported. For ACM, n is set to 3, and the number of encrypted layers is 6 for ViT-B and 7 for ViT-L.

Method	Time (s) on ViT-B	Time (s) on ViT-L	Device
AES-CBC	2.30	9.02	CPU
AES-CTR	1.34	5.17	CPU
AES-GCM	1.45	5.18	CPU
TEA	19.63	69.51	CPU
XOR Cipher	3.72	13.47	CPU
Our Method (CPU)	**1.19**	**2.58**	CPU
XOR Cipher (GPU)	0.043	0.22	GPU
Our Method (GPU)	**0.044**	**0.051**	GPU

5.4.2 Comparison with Other IP Protection Approaches. Tab. 7 compares our proposed work with representative IP protection approaches in the literature. In particular, the "Retrain" column denotes whether the model owner needs to retrain the models with additional regularizers or customized datasets, which often impede convergence and prolong the training process while also lacking scalability for a substantial number of users. In contrast, like all encryption-based schemes, our approach does not require retraining or hurt model performance ("Acc. Loss" column).

In terms of "Inference Overhead", though passive strategies typically do not introduce inference-time overhead, they cannot offer protection but only provide ownership tracing. TransLinkGuard (TLG) [59] addresses on-device transformer deployment by securing layers with weight matrix permutations within a TEE. However, TLG is ill-suited for real-time edge deployment, as its mask–offload–restore pipeline requires the TEE to repeatedly compute noise effects mW (matrix multiplication), which incurs prohibitive latency and memory overhead [86]. Moreover, their obfuscation scheme is purely linear, and attackers with access to permuted weights and the original public model might align columns to recover the permutations [108]. For methods that do not explicitly report inference-time overhead [26, 61], we evaluate their publicly available implementations under the same hardware and software configuration as ours, using standard ResNet-18 models with ImageNet-scale inputs (224×224) and a consistent batch size (e.g., 64). As demonstrated in our comparative analysis, `ChaosFormer` achieves substantially lower inference overhead than all evaluated prior works, establishing it as the most efficient active defense for edge devices. While `ChaosFormer-S` incurs a slightly higher computational cost to provide enhanced statistical security, its overhead remains significantly lower than existing alternatives.

5.5 Ablation Study

5.5.1 Impact of Encrypting FFN. We perform an ablation study on the encryption of the FFN layer in the transformer layers. Tab. 8 reports the effectiveness of applying ACM encryption *only* to the attention weights. The experimental results show that significantly more layers must be encrypted

Table 7. Comparison of prior IP protection methods.

Work	Type	Mechanism	Retrain	Target	Infer. Overhead	Acc. Loss
Uchida et al. [104]	Passive	WB Watermarking	✓	CNN	None	✓
Adi et al. [1]	Passive	BD Watermarking	✓	CNN	None	✓
M-LOCK [84]	Active	BD Access Control	✓	CNN	None	✓
DeepIPR [26]	Active	Passport Verification	✓	CNN	Moderate (+50%)	✓
NNLock [2]	Active	Wt. Encrption	×	CNN	Moderate (+100%)	×
Phantom [3]	Active	Arch. Obfusc., TEE	✓♯	CNN	Moderate (+250%♮1)	✓♮2
ChaoW [61]	Active	Wt. Encryption	×	CNN	Moderate (+105%)	×
Full TEE [114]	Active	Full Exec. in TEE	×	TF	Very High (∼112×)	×
TLG [59]	Active	Wt. Permutation, TEE	×	TF	High [86]	×
LoRO [114]	Active	Wt. Permutation, TEE	×	TF	Moderate (+149%)	×
ChaosFormer	**Active**	**Wt. Encryption, PUF**	×	**TF**	**Low (+2–10%)***	×
ChaosFormer-S	**Active**	**Wt. Encryption, PUF**	×	**TF**	**Low (+7–26%)**	×

Abbreviations: WB: White-box; BD: Backdoor; Wt.: Weight; Exec.: Execution; Arch. Obfusc.: Architecture Obfuscation; CNN: Convolutional Neural Network; TF: Transformer; Infer.: Inference. ♯Includes neural architecture search (≈ 26h for ResNet-18 on CIFAR-10, single A6000). ♮1 Includes obfuscation layer, TEE latency, and TEE–GPU communication; with ResNet-18, SGX2, Top-3 strategy. ♮2 No direct accuracy loss, but uses 8-bit quantization for TEE–GPU data transfer. *Based on encrypting 6 layers.

to reach a comparable model degradation effect when the FFN is left untouched. As demonstrated in sec. 5.3, encrypting more layers introduces additional overhead, while skipping the FFN does not measurably reduce inference latency. Therefore, our proposed method of encrypting both the attention and FFN weights is necessary to obtain a strong locking effect at minimal cost.

Table 8. Model accuracy (%) when encrypting only attention weights. The number in parentheses indicates the layer index. Encrypting only attention weights requires more layers to achieve satisfactory performance degradation.

Steps	ViT-B	ViT-B-384	ViT-B-8b	BeiT-B	DeiT-B	Swin-B
Acc_0 (%)	80.32	83.91	80.24	84.54	80.20	85.14
1	56.46 (11)	60.65 (11)	53.64 (11)	62.67 (1)	75.69 (10)	64.36 (11)
2	13.87 (10)	14.37 (10)	11.35 (10)	8.23 (2)	31.33 (11)	42.16 (10)
3	4.87 (9)	4.83 (9)	3.93 (9)	0.87 (3)	1.04 (9)	12.01 (9)
4	2.01 (8)	2.11 (8)	1.61 (8)	0.50 (6)	0.15 (8)	2.09 (8)
5	0.97 (7)	1.01 (0)	0.74 (7)	0.30 (8)	0.10 (7)	0.63 (7)
6	0.46 (0)	0.45 (7)	0.30 (0)	0.13 (5)	0.09 (5)	0.30 (0)
7	0.21 (6)	0.22 (6)	0.20 (6)	—	—	0.12 (1)
8	0.17 (5)	0.13 (5)	0.14 (5)	—	—	0.11 (6)

5.5.2 Impact of Extra Security Layers. To empirically validate our claim that adding extra security layers enhances robustness, we conduct a controlled experiment isolating this variable. We perform retraining attacks on ViT-B models obfuscated with a varying number of additional security layers, from zero to four. For each case, the attacker's best attempt at recovering performance by retraining the model's full parameters with 20% of the original training dataset is recorded.

The results, presented in tab. 9, demonstrate a clear trend: increasing the number of randomly selected extra security layers significantly degrades the model's post-retraining performance. This enhances the robustness of the IP protection, as it becomes progressively harder for an attacker to restore the model's functionality. We observe that after adding three extra layers, the attack accuracy begins to plateau, indicating that a small number of additional layers is sufficient to provide a strong defense. In practice, the number of extra layers can be chosen to balance this enhanced security against a minor increase in inference overhead.

Table 9. Impact of extra security layers on retraining ViT-B. The baseline model required two layers to fall below the random-guess threshold; the "Number of Extra Security Layers" column refers to layers added on top of these initial two.

Number of Extra Security Layers	Best Top-1 Acc. after Retraining (%)
0	69.7
1	68.1
2	58.4
3	54.5
4	56.4

6 Related Works

This section reviews IP protection methods for deep learning models. We hereby categorize previous efforts into **passive protection** and **proactive protection**, and within each category, we discuss the major sub-families and how `ChaosFormer` relates to them.

6.1 Passive Protection of DNN

Passive protection (sometimes called reactive protection) aims to enable model owners to *prove* prior possession after a theft has occurred, rather than to prevent the theft itself.

White-box watermarking. The earliest line of work assumes that the dispute resolver has access to the suspect model's parameters. Uchida et al. [104] embed an owner-specific signature into the weights via an additional regularizer; DeepSigns [18] extends this to embedding into the activation distributions of selected layers; PCPT/ACPT [27] adds a copyright tracing layer that supports public verification. White-box methods give crisp ownership signals but require the verifier to physically access the disputed weights, which is exactly the scenario our hardware-bound binding makes obsolete.

Black-box / trigger-set watermarking. A second line of work embeds a backdoor-style trigger set into the model: certain crafted inputs produce attacker-chosen outputs that the legitimate owner can use to claim ownership. Adi et al. [1] pioneered this construction; Zhang et al. [122] apply it to commercial DNN services; Liu et al. [64] embed the trigger as a greedy residual; Li et al. [56] design watermarks resistant to fine-tuning; DeepFidelity [41] studies the fidelity-robustness trade-off; explanation-based watermarks [88] use feature-attribution patterns. Recent work also explores training-free post-deployment watermarks [17, 43]. The fundamental limitation of black-box watermarking is that the verifier must be able to query the suspect model, which a sophisticated adversary running the stolen model privately can simply refuse.

Fingerprinting. Fingerprinting extracts a unique identifier from the target model rather than embedding one. Sub-families include adversarial-example fingerprints [128], decision-boundary fingerprints [8], characteristic-example fingerprints [109], intrinsic fingerprints from edge-device behavior [80], non-repudiable fingerprints [129], and recent robustness-based extensions [116].

Like watermarking, fingerprinting is reactive and requires either the suspect model's parameters (white-box) or its query interface (black-box).

Robustness of passive schemes. A growing body of work demonstrates that passive schemes are fragile against adaptive adversaries. REFIT [15] shows watermark removal via fine-tuning under limited data; Shafieinejad et al. [87] attack backdoor watermarks; Sun et al. [93] remove watermarks via GANs; Yan et al. [117] break white-box watermarks under structural obfuscation; DeepEclipse [81] provides a unified attack framework; Pang et al. [79] and Jovanović et al. [49] show watermark stealing in LLMs. Despite these limitations, watermarking and fingerprinting are still useful as a complement to the proactive protection we propose: `ChaosFormer` prevents misuse on unauthorized devices, while a watermark or fingerprint can support post-hoc legal proceedings if a leak does occur.

6.2 Proactive Protection

Proactive schemes aim to block unauthorized inference from the outset, typically by requiring a correct key, license, or trusted hardware path. We organise them by mechanism.

Encryption-based methods. Several works lock neural networks with cryptographic keys without modifying the training process. NNLock [2] encrypts individual parameters with a lightweight scheme but incurs substantial CPU overhead. LEWIP [72] surveys recent DNN IP protection schemes and breaks several of them, motivating the need for formal security guarantees of the kind we provide via SPI. Goldstein et al. [33] build an obfuscation pipeline backed by a hardware security module. CoreLocker [110] and NNSplitter [130] identify a small "critical-weight" subset (often less than 1% of parameters) and protect only those weights; this minimal-perturbation strategy was shown by Sun et al. [94] to expose statistical fingerprints that an adaptive attacker can exploit, which is precisely the vulnerability that `ChaosFormer` (and especially ChaosFormer-S) is designed to avoid by encrypting whole layers and randomising their distribution.

PUF-based DNN protection. Using PUFs to bind applications to a specific device is a well-established idea in cryptographic hardware [7]. The use of PUFs specifically for DNN protection is more recent. Lin et al. [61] were the first to combine ACM-style chaotic permutation with hardware binding for CNNs and RNNs; Goldstein et al. [33] explore a related HSM-backed approach; PIPP [54] uses PUFs for FPGA-deployed DNN IP protection; Pan et al. [78] present a permute-diffusion scheme for MLPs and CNNs; Jiang et al. [47] add publicly verifiable ownership; Rajendran et al. [82] target binarized neural networks via PUF-based key management in memristive crossbars; Guo et al. [35] present an early pay-per-device CNN IP-protection scheme. None of these works specifically address transformers — their encryption strategies, designed for CNN convolution kernels or MLP weight matrices, do not naturally fit attention layers (square matrices, quadratic in head dimension) or FFN layers (rectangular, 4× embedding dimension). `ChaosFormer` is the first PUF-based scheme designed specifically for the transformer architecture, with separate primitives for attention (ACM) and FFN (Knuth shuffle) and formal SPI security in the diffusion-augmented variant.

TEE-based methods. TEEs protect model parameters during runtime by partitioning the inference into a trusted secure enclave and an untrusted accelerator. Foundational works include oblivious training on SGX [74], federated learning with TEEs [69], and DarkneTZ [70], which uses TrustZone for partial DNN inference. ShadowNet [95] and Occlumency [53] target on-device CNN inference; SOTER [89] guards black-box inference at the edge; MirrorNet [66] and ASGARD [71] extend TEE-based inference with virtualization. For transformers specifically, TransLinkGuard [59] secures layer permutations within a TEE but pays for it with high inference latency; LoRO [114] uses low-rank obfuscation; Phantom [3] obfuscates the model architecture. DeepContract [131] uses TEEs to support remote license revocation. The fundamental issue with TEE-based schemes is the limited secure memory: 128 MB in Intel SGX1 and up to 16 MB for most ARM TrustZone

devices, which is well below the size of modern transformer models, leading to memory swapping that dominates inference time [107, 127]. Side-channel work on TEEs [9, 16, 52, 120, 121, 124] has further demonstrated that the boundary between trusted and untrusted regions is itself an attack surface.

GPU-side TEE designs. Because the bulk of transformer computation happens on the GPU, several proposals extend TEE-style guarantees to the GPU. Graviton [106] requires hardware modifications to the GPU; HIX [45] extends SGX with heterogeneous device protection; GEVisor [112] and Honeycomb [68] provide GPU isolation through software adaptation. All of these approaches share a deployment cost: either the GPU silicon must change (typically a multi-year cycle) or substantial system-software work is required to adapt the application stack. `ChaosFormer` sidesteps this entirely by operating purely on the model file with PUF-derived keys, leaving the inference path on the GPU unchanged.

Model-level access control. A different family of proactive schemes embeds access control directly into the model's behavior, granting full functionality only to users who possess specific secret information. Chen and Wu [14] add an active control module that disables inference under adversarial perturbations; DeepIPR [26] introduces passport layers whose scale and bias factors are correct only when a designated passport input is provided; M-LOCK [84] uses backdoor-style data poisoning for cybersecurity services; SecureNet [58] uses backdoor learning for proactive IP protection; AdvParams [115] encrypts parameters via adversarial perturbations; PSS [99] performs probabilistic selective encryption for hierarchical services; EdgePro [12] authorizes individual neurons. The common drawback is that these schemes require retraining or fine-tuning with extra objectives, which becomes prohibitively expensive for transformer-scale models and lacks scalability when separate models must be issued per user.

Memory-extraction attacks. Several recent works demonstrate that runtime memory is a real attack surface for unprotected models. Sun et al. [96] survey insufficient model protection in mobile apps at scale; Deng et al. [21] characterize on-device DL model threats in Android; Oygenblik et al. [77] reconstruct full models from process-memory dumps; Ren et al. [85] identify model-stealing vulnerabilities in deployed mobile apps; Liu et al. [67] decompile DNN models from binaries; cold-boot attacks [34, 36, 62, 73] show that DRAM contents survive briefly after power-down, enabling weight extraction; cache-based side channels [63, 83, 119] can leak weights bit-by-bit. ChaosFormer's layer-by-layer dynamic decryption (sec. 5.3) addresses these threats by ensuring that the full plaintext model is never simultaneously resident in memory.

Architecture-stealing attacks. A separate line of work targets the model's architecture rather than its weights. DeepSniffer [40] infers architecture from execution traces; Zhang et al. [126] use FPGA side-channels; DeepTheft [31] steals architectures via power side-channels; NeurObfuscator [57] provides architectural obfuscation. `ChaosFormer` assumes the architecture is public (the standard assumption for transformer-family models), but as we note in sec. 7, architectural obfuscation is a natural complement to weight encryption.

Our focus. Most of the solutions above either focus on small networks (CNNs, MLPs), require retraining, depend on TEE primitives, or assume an attacker without partial training data and reference checkpoints. In contrast, `ChaosFormer` encrypts transformer models using PUF-driven keys without retraining, avoids TEE constraints, and provides robust defense under the white-box plus public-checkpoint setting described in sec. 3. To our knowledge, it is the first PUF-based active protection scheme designed specifically for transformers and the first to come with a formal SPI security guarantee.

6.3 Limitations and Scope

Trust in the hardware vendor. Our protocol (sec. 4.1) assumes the hardware vendor faithfully enrolls the PUF, securely stores the CRP database, and removes the readout interface after registration. A malicious vendor could, in principle, leak CRPs and thereby allow attackers to derive valid keys. This is a structural property of any PUF-based scheme rather than a flaw of `ChaosFormer` specifically; mitigations include third-party-audited enrollment, distributed CRP storage with secret sharing, and post-enrollment proof-of-destruction of the readout interface.

PUF aging and reliability. Silicon-based PUFs may drift over the device lifetime due to temperature, voltage, and electromigration effects, potentially causing the response R to differ from its registered value. We rely on the fuzzy-extractor framework [10, 19, 50] with helper data and error-correcting codes to recover a stable key, which is standard practice for production PUF deployments. The reliability budget of the chosen fuzzy extractor sets a hard cap on tolerable noise, and devices that drift beyond this cap will fail to decrypt; in such cases, the user must re-enroll the device, which is a known operational cost of PUF-based systems.

Excluded attack classes. As stated in sec. 3, our threat model excludes invasive hardware attacks (decapping, focused-ion-beam probing) and direct side-channel attacks on the PUF circuitry itself (electromagnetic, photonic). Defending against these requires hardware-level countermeasures (shielding, sensor meshes, masking) that are orthogonal to the algorithmic protections we provide. We also exclude indirect black-box model extraction through query-based substitute training [75, 76, 103], which requires a separate defense layer on the inference API [28, 55].

Scaling to billion-parameter models. Our experiments validate `ChaosFormer` on vision transformers of up to 307M parameters (ViT-L). For models in the 7B–70B range (e.g., LLaMA-class LLMs), the layer-by-layer dynamic encryption pattern carries the same asymptotic overhead, but the absolute cost of moving large layer tensors through the encryption kernel grows. We discuss this as a direction for future work in sec. 7; preliminary back-of-envelope analysis suggests that aggressive layer selection (encrypting $K \ll L_{\text{total}}$ layers) and the on-the-fly keystream optimization (sec. 4.2.3) keep the overhead manageable, but a full evaluation is left to a future study.

Architecture privacy. `ChaosFormer` protects the weights but assumes that the network architecture is public knowledge — a reasonable assumption for transformer-family models, where the architecture is essentially standardized. If architectural privacy is also required, `ChaosFormer` should be combined with an architectural-obfuscation layer [3, 57], which is one of the future directions discussed in our conclusion.

This manuscript substantially extends our preliminary work [42]. The primary new contributions include: (1) the introduction of `ChaosFormer-S` (sec. 4.2), a diffusion-augmented variant ensuring that encrypted weights are statistically indistinguishable from random noise; (2) rigorous formal security proofs (e.g., Single-Point Indistinguishability) for the framework (sec. 4.2.1); (3) extended robustness evaluations against domain-transfer attacks on complex datasets (OfficeHome, DomainNet) (sec. 5.2); (4) detailed edge-deployment optimizations, including side-channel-resilient execution and fused GPU kernels (sec. 4.2.3); (5) a hardware-bound ownership-verification protocol (sec. 4.4) that supports public verification without database access; and (6) a substantially expanded discussion of related work, deployment scenarios, and limitations.

7 Conclusion

We propose an active, hardware-bound IP protection scheme that integrates PUF-based cryptographic key derivation, ACM-based weight obfuscation, and secure diffusion for ultimate security. The resulting model cannot be executed correctly outside the authorized device, effectively thwarting IP theft and mitigating attacks that rely on stolen models, even under white-box settings. Unlike

watermarking or retraining-based schemes, our method imposes no changes to the original training process and adds minimal runtime overhead.

Looking ahead, there are three clear paths to strengthen and scale this protection. First, while our current work focuses on Transformers, the core framework is adaptable to other architectures, such as CNNs. For 4D convolutional kernels, which are typically not square, one could apply a rectangular chaotic map such as the Baker's Map [30] in place of ACM. Second, we plan to extend and adapt our layer-selection heuristic to billion-parameter models (e.g., Large Language Models) so that overhead remains manageable even at that scale. Third, while our method only protects the weights and assumes full knowledge of the architecture, we can further integrate architectural obfuscation [3, 57] to achieve deeper protection against architecture-stealing attacks [31, 40, 126]. By offering a low-cost, framework-agnostic lock on valuable model IP, our approach takes a practical step toward securing on-device deep learning, from edge applications to large-scale deployments.

Acknowledgments

This work was supported by the National Natural Science Foundation of China [grant number 62404192] and the Shenzhen Stability Science Program 2023.

References

[1] Yossi Adi, Carsten Baum, Moustapha Cissé, Benny Pinkas, and Joseph Keshet. 2018. Turning Your Weakness Into a Strength: Watermarking Deep Neural Networks by Backdooring. In *Proc. 27th USENIX Secur. Symp.* USENIX Association, 1615–1631.

[2] Manaar Alam, Sayandeep Saha, Debdeep Mukhopadhyay, and Sandip Kundu. 2022. *NN-Lock*: A Lightweight Authorization to Prevent IP Threats of Deep Learning Models. *ACM J. Emerg. Technol. Comput. Syst.* 18, 3 (2022), 51:1–51:19. doi:10.1145/3505634

[3] Juyang Bai, Md Hafizul Islam Chowdhuryy, Jingtao Li, Fan Yao, Chaitali Chakrabarti, and Deliang Fan. 2025. Phantom: Privacy-Preserving Deep Neural Network Model Obfuscation in Heterogeneous TEE and GPU System. In *Proc. 34th USENIX Secur. Symp.* USENIX Association, 5565–5582.

[4] Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. 2022. BEiT: BERT Pre-Training of Image Transformers. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022.* OpenReview.net. https://openreview.net/forum?id=p-BhZSz59o4

[5] Mihir Bellare, Thomas Ristenpart, Phillip Rogaway, and Till Stegers. 2009. Format-Preserving Encryption. In *Selected Areas in Cryptography*, Vol. 5867. Springer, 295–312. doi:10.1007/978-3-642-05445-7_19

[6] Daniel J Bernstein et al. 2008. ChaCha, a variant of Salsa20. In *Workshop record of SASC*, Vol. 8. Lausanne, Switzerland, 3–5.

[7] Christoph Bösch, Jorge Guajardo, Ahmad-Reza Sadeghi, Jamshid Shokrollahi, and Pim Tuyls. 2008. Efficient Helper Data Key Extractor on FPGAs. In *Cryptogr. Hardware Embedded Syst. (CHES) (Lecture Notes in Computer Science, Vol. 5154)*. Springer, 181–197. doi:10.1007/978-3-540-85053-3_12

[8] Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. 2021. IPGuard: Protecting intellectual property of deep neural networks via fingerprinting the classification boundary. In *Proc. ACM Asia Conf. Comput. Commun. Secur.* 14–25.

[9] David Cerdeira, Nuno Santos, Pedro Fonseca, and Sandro Pinto. 2020. SoK: Understanding the Prevailing Security Vulnerabilities in TrustZone-assisted TEE Systems. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, USA, 1416–1432. doi:10.1109/SP40000.2020.00061

[10] Chip-Hong Chang, Yue Zheng, and Le Zhang. 2017. A retrospective and a look forward: Fifteen years of physical unclonable function advancement. *IEEE Circuits Syst. Mag.* 17, 3 (2017), 32–62. doi:10.1109/MCAS.2017.2713305

[11] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM Trans. Intell. Syst. Technol.* 15, 3 (2024), 1–45.

[12] Jinyin Chen, Haibin Zheng, Tao Liu, Jiawei Liu, Yao Cheng, Xuhong Zhang, and Shouling Ji. 2024. EdgePro: Edge Deep Learning Model Protection via Neuron Authorization . *IEEE Trans. Dependable and Secure Computing* 21, 05 (2024), 4967–4981. doi:10.1109/TDSC.2024.3365730

[13] Lei Chen and Shihong Wang. 2015. Differential Cryptanalysis of a Medical Image Cryptosystem with Multiple Rounds. *Computers in Biology and Medicine* 65 (2015), 69–75. doi:10.1016/j.compbiomed.2015.07.024

[14] Mingliang Chen and Min Wu. 2018. Protect Your Deep Neural Networks from Piracy. In *2018 IEEE International Workshop on Information Forensics and Security (WIFS)*. IEEE, Hong Kong, China, 1–7. doi:10.1109/WIFS.2018.8630791

[15] Xinyun Chen, Wenxiao Wang, Chris Bender, Yiming Ding, Ruoxi Jia, Bo Li, and Dawn Song. 2021. REFIT: A Unified Watermark Removal Framework For Deep Learning Systems With Limited Data. In *ACM Asia Conf. Comput. Commun. Secur.* ACM, 321–335. doi:10.1145/3433210.3453079

[16] Victor Costan and Srinivas Devadas. 2016. Intel SGX explained. *Cryptology ePrint Archive* (2016).

[17] Kieu Dang, Phung Lai, NhatHai Phan, Yelong Shen, Ruoming Jin, Abdallah Khreishah, and My Thai. 2025. SoK: Are Watermarks in LLMs Ready for Deployment? *ArXiv preprint* abs/2506.05594 (2025). https://arxiv.org/abs/2506.05594

[18] Bita Darvish Rouhani, Huili Chen, and Farinaz Koushanfar. 2019. Deepsigns: An end-to-end watermarking framework for ownership protection of deep neural networks. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 485–497.

[19] Jeroen Delvaux, Dawu Gu, Ingrid Verbauwhede, Matthias Hiller, and Meng-Day Yu. 2016. Efficient fuzzy extraction of PUF-induced secrets: Theory and applications. In *International Conference on Cryptographic Hardware and Embedded Systems*. Springer, 412–431.

[20] Zizhuang Deng, Kai Chen, Guozhu Meng, Xiaodong Zhang, Ke Xu, and Yao Cheng. 2022. Understanding Real-world Threats to Deep Learning Models in Android Apps. In *ACM SIGSAC Conf. Comput. Commun. Secur.* ACM, 785–799. doi:10.1145/3548606.3559388

[21] Zizhuang Deng, Kai Chen, Guozhu Meng, Xiaodong Zhang, Ke Xu, and Yao Cheng. 2022. Understanding Real-world Threats to Deep Learning Models in Android Apps. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (Los Angeles, CA, USA) *(CCS '22)*. Association for Computing Machinery, New York, NY, USA, 785–799. doi:10.1145/3548606.3559388

[22] Giuseppe Desolda, Lauren S. Ferro, Andrea Marrella, Tiziana Catarci, and Maria Francesca Costabile. 2021. Human Factors in Phishing Attacks: A Systematic Literature Review. *ACM Comput. Surv.* 54, 8, Article 173 (2021), 35 pages. doi:10.1145/3469886

[23] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *Proc. of ICLR*. OpenReview.net. https://openreview.net/forum?id=YicbFdNTTy

[24] Abdulrahman Abu Elkhail, Anys Bacha, and Hafiz Malik. 2025. Sniper: Countering Locker Ransomware Attacks Through Natural Language Processing. *IEEE Trans. Dependable Secur. Comput.* 22, 4 (2025), 4160–4175. doi:10.1109/TDSC.2025.3543268

[25] Mahmoud F. Abd Elzaher, Mohamed Shalaby, and Salwa H. El Ramly. 2016. An Arnold Cat Map-Based Chaotic Approach for Securing Voice Communication. In *Proceedings of the 10th International Conference on Informatics and Systems*. ACM, Giza Egypt, 329–331. doi:10.1145/2908446.2908508

[26] Lixin Fan, Kam Woh Ng, Chee Seng Chan, and Qiang Yang. 2022. DeepIPR: Deep Neural Network Ownership Verification With Passports. *IEEE Trans. Pattern Anal. Mach. Intell.* 44, 10 (2022), 6122–6139. doi:10.1109/TPAMI.2021.3088846

[27] Xuefeng Fan, Dahao Fu, Hangyu Gui, Xinpeng Zhang, and Xiaoyi Zhou. 2022. PCPT and ACPT: Copyright Protection and Traceability Scheme for DNN Models. https://arxiv.org/abs/2206.02541

[28] Ryan Feng, Ashish Hooda, Neal Mangaokar, Kassem Fawaz, Somesh Jha, and Atul Prakash. 2023. Stateful Defenses for Machine Learning Models Are Not Yet Secure Against Black-box Attacks. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) *(CCS '23)*. Association for Computing Machinery, New York, NY, USA, 786–800. doi:10.1145/3576915.3623116

[29] Jiri Fridrich. 1998. Symmetric Ciphers Based on Two-Dimensional Chaotic Maps. *Int. J. Bifurcation and Chaos* 08, 06 (1998), 1259–1284. doi:10.1142/S021812749800098X

[30] Jiri Fridrich. 1998. Symmetric ciphers based on two-dimensional chaotic maps. *Int. J. Bifurcat. Chaos* 8, 06 (1998), 1259–1284.

[31] Yansong Gao, Huming Qiu, Zhi Zhang, Binghui Wang, Hua Ma, Alsharif Abuadbba, Minhui Xue, Anmin Fu, and Surya Nepal. 2024. DeepTheft: Stealing DNN Model Architectures through Power Side Channel. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 3311–3326. doi:10.1109/SP54263.2024.00250

[32] Blaise Gassend, Marten van Dijk, Dwaine E. Clarke, Emina Torlak, Srinivas Devadas, and Pim Tuyls. 2008. Controlled physical random functions and applications. *ACM Trans. Inf. Syst. Secur.* 10, 4 (2008), 3:1–3:22. doi:10.1145/1284680.1284683

[33] Brunno F. Goldstein, Vinay C. Patil, Victor C. Ferreira, Alexandre S. Nery, Felipe M. G. França, and Sandip Kundu. 2021. Preventing DNN Model IP Theft via Hardware Obfuscation. *IEEE J. Emerging and Selected Topics in Circuits and Systems* 11, 2 (2021), 267–277.

[34] Michael Gruhn and Tilo Müller. 2013. On the Practicability of Cold Boot Attacks. In *2013 International Conference on Availability, Reliability and Security*. IEEE Computer Society, Washington, DC, USA, 390–397. doi:10.1109/ARES.2013.52

[35] Qingli Guo, Jing Ye, Yue Gong, Yu Hu, and Xiaowei Li. 2018. PUF Based Pay-Per-Device Scheme for IP Protection of CNN Model. In *Proc. 2018 IEEE 27th Asian Test Symposium (ATS)*. IEEE, Hefei, China, 115–120. doi:10.1109/ATS.2018.00032 Presented at the 2018 IEEE 27th Asian Test Symposium (ATS), October 15–18, Hefei, China.

[36] J. Alex Halderman, Seth D. Schoen, Nadia Heninger, William Clarkson, William Paul, Joseph A. Calandrino, Ariel J. Feldman, Jacob Appelbaum, and Edward W. Felten. 2009. Lest we remember: cold-boot attacks on encryption keys. *Commun. ACM* 52, 5 (2009), 91–98. doi:10.1145/1506409.1506429

[37] Kai Han, Yunhe Wang, Hanting Chen, Xinghao Chen, Jianyuan Guo, Zhenhua Liu, Yehui Tang, An Xiao, Chunjing Xu, Yixing Xu, et al. 2022. A survey on vision transformer. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 1 (2022), 87–110.

[38] Grant Ho, Asaf Cidon, Lior Gavish, Marco Schweighauser, Vern Paxson, Stefan Savage, Geoffrey M. Voelker, and David Wagner. 2019. Detecting and Characterizing Lateral Phishing at Scale. In *Proceedings of the 28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, USA, 1273–1290. https://www.usenix.org/conference/usenixsecurity19/presentation/ho

[39] Daniel E. Holcomb, Wayne P. Burleson, and Kevin Fu. 2009. Power-Up SRAM State as an Identifying Fingerprint and Source of True Random Numbers. *IEEE Trans. Computers* 58, 9 (2009), 1198–1210. doi:10.1109/TC.2008.212

[40] Xing Hu, Ling Liang, Shuangchen Li, Lei Deng, Pengfei Zuo, Yu Ji, Xinfeng Xie, Yufei Ding, Chang Liu, Timothy Sherwood, et al. 2020. Deepsniffer: A dnn model extraction framework based on learning architectural hints. In *Proc. 25th Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*. 385–399.

[41] Guang Hua and Andrew Beng Jin Teoh. 2023. Deep fidelity in DNN watermarking: A study of backdoor watermarking for classification models. *Pattern Recognit.* 144 (2023), 109844. doi:10.1016/J.PATCOG.2023.109844

[42] Peichun Hua, Hanxiu Zhang, Tuo Li, and Yue Zheng. 2025. Securing On-device Transformer with Hardware Binding and Reversible Obfuscation. In *IEEE Annual Computer Security Applications Conference, ACSAC 2025, Honolulu, HI, USA, December 8-12, 2025*. IEEE, 956–972. doi:10.1109/ACSAC67867.2025.00079

[43] Yujin Huang, Zhi Zhang, Qingchuan Zhao, Xingliang Yuan, and Chunyang Chen. 2025. THEMIS: Towards Practical Intellectual Property Protection for Post-Deployment On-Device Deep Learning Models. In *34th USENIX security symposium (USENIX Security 25)*.

[44] Usman Inayat, Mashaim Farzan, Sajid Mahmood, Muhammad Fahad Zia, Shahid Hussain, and Fabiano Pallonetto. 2024. Insider threat mitigation: Systematic literature review. *Ain Shams Engineering Journal* (2024). doi:10.1016/j.asej.2024.103068

[45] Insu Jang, Adrian Tang, Taehoon Kim, Simha Sethumadhavan, and Jaehyuk Huh. 2019. Heterogeneous isolated execution for commodity gpus. In *Proc. 24th Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*. 455–468.

[46] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *ArXiv preprint* abs/2406.00515 (2024). https://arxiv.org/abs/2406.00515

[47] Jingdong Jiang, Yue Zheng, and Chip-Hong Chang. 2025. PUF-based Edge DNN Model IP Protection with Self-obfuscation and Publicly Verifiable Ownership. In *IEEE International Symposium on Circuits and Systems, ISCAS 2025, London, United Kingdom, May 25-28, 2025*. IEEE, 1–5. doi:10.1109/ISCAS56072.2025.11044032

[48] Zhengyuan Jiang, Minghong Fang, and Neil Zhenqiang Gong. 2023. Ipcert: Provably robust intellectual property protection for machine learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 3612–3621. doi:10.1109/ICCVW60793.2023.00389

[49] Nikola Jovanovic, Robin Staab, and Martin T. Vechev. 2024. Watermark Stealing in Large Language Models. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. https://openreview.net/forum?id=Wp054bnPq9

[50] Hyunho Kang, Yohei Hori, Toshihiro Katashita, Manabu Hagiwara, and Keiichi Iwamura. 2014. Cryptographie key generation from PUF data using efficient fuzzy extractors. In *16th International conference on advanced communication technology*. IEEE, 23–26.

[51] Jeremie S. Kim, Minesh Patel, Hasan Hassan, and Onur Mutlu. 2018. The DRAM Latency PUF: Quickly Evaluating Physical Unclonable Functions by Exploiting the Latency-Reliability Tradeoff in Modern Commodity DRAM Devices. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Washington, DC, USA, 194–207. doi:10.1109/HPCA.2018.00026

[52] Zili Kou, Sharad Sinha, Wenjian He, and Wei Zhang. 2023. Cache Side-channel Attacks and Defenses of the Sliding Window Algorithm in TEEs. In *Proc. 2023 Design, Automat. Test Europe Conf. Exhib. (DATE)*. IEEE, Antwerp, Belgium, 1–6. doi:10.23919/DATE56975.2023.10137116

[53] Taegyeong Lee, Zhiqi Lin, Saumay Pushp, Caihua Li, Yunxin Liu, Youngki Lee, Fengyuan Xu, Chenren Xu, Lintao Zhang, and Junehwa Song. 2019. Occlumency: Privacy-preserving remote deep-learning inference using SGX. In *The 25th Annual international conference on mobile computing and networking*. 1–17.

[54] Dawei Li, Yangkun Ren, Di Liu, Yuxiao Guo, Zhenyu Guan, and Jianwei Liu. 2024. PIPP: A Practical PUF-Based Intellectual Property Protection Scheme for DNN Model on FPGA. *IEEE Trans. Circuits Syst. II Express Briefs* 71, 2 (2024), 912–916. doi:10.1109/TCSII.2023.3309105

[55] Huiying Li, Shawn Shan, Emily Wenger, Jiayun Zhang, Haitao Zheng, and Ben Y. Zhao. 2022. Blacklight: Scalable Defense for Neural Networks against Query-Based Black-Box Attacks. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 2117–2134. https://www.usenix.org/conference/usenixsecurity22/presentation/li-huiying

[56] Huiying Li, Emily Wenger, Shawn Shan, Ben Y. Zhao, and Haitao Zheng. 2019. Piracy Resistant Watermarks for Deep Neural Networks. https://arxiv.org/abs/1910.01226

[57] Jingtao Li, Zhezhi He, Adnan Siraj Rakin, Deliang Fan, and Chaitali Chakrabarti. 2021. Neurobfuscator: A full-stack obfuscation tool to mitigate neural architecture stealing. In *Proc. IEEE Int. Symp. Hardw. Oriented Secur. Trust (HOST)*. IEEE, 248–258.

[58] Peihao Li, Jie Huang, Huaqing Wu, Zeping Zhang, and Chunyang Qi. 2024. SecureNet: Proactive intellectual property protection and model security defense for DNNs based on backdoor learning. *Neural Netw.* 174 (2024), 106199. doi:10.1016/J.NEUNET.2024.106199

[59] Qinfeng Li, Zhiqiang Shen, Zhenghan Qin, Yangfan Xie, Xuhong Zhang, Tianyu Du, Sheng Cheng, Xun Wang, and Jianwei Yin. 2024. TransLinkGuard: Safeguarding Transformer Models Against Model Stealing in Edge Deployment. In *Proc. 32nd ACM Int. Conf. Multimedia*. ACM, 3479–3488. doi:10.1145/3664647.3680786

[60] Daihyun Lim, Jae W. Lee, Blaise Gassend, G. Edward Suh, Marten van Dijk, and Srinivas Devadas. 2005. Extracting secret keys from integrated circuits. *IEEE Trans. Very Large Scale Integr. Syst.* 13, 10 (2005), 1200–1205. doi:10.1109/TVLSI.2005.859470

[61] Ning Lin, Xiaoming Chen, Hang Lu, and Xiaowei Li. 2021. Chaotic Weights: A Novel Approach to Protect Intellectual Property of Deep Neural Networks. *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems* 40, 7 (2021), 1327–1339.

[62] Simon Lindenlauf, Hans Höfken, and Marko Schuba. 2015. Cold Boot Attacks on DDR2 and DDR3 SDRAM. In *2015 10th International Conference on Availability, Reliability and Security (ARES)*. IEEE Computer Society, Washington, DC, USA, 287–292. doi:10.1109/ARES.2015.28

[63] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B. Lee. 2015. Last-Level Cache Side-Channel Attacks Are Practical. In *2015 IEEE Symposium on Security and Privacy*. IEEE, San Jose, CA, 605–622. doi:10.1109/SP.2015.43

[64] Hanwen Liu, Zhenyu Weng, and Yuesheng Zhu. 2021. Watermarking Deep Neural Networks with Greedy Residuals. In *Proc. of ICML (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 6978–6988. http://proceedings.mlr.press/v139/liu21x.html

[65] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021*. IEEE, 9992–10002. doi:10.1109/ICCV48922.2021.00986

[66] Ziyu Liu, Yukui Luo, Shijin Duan, Tong Zhou, and Xiaolin Xu. 2023. Mirrornet: A tee-friendly framework for secure on-device dnn inference. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.

[67] Zhibo Liu, Yuanyuan Yuan, Shuai Wang, Xiaofei Xie, and Lei Ma. 2023. Decompiling x86 deep neural network executables. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 2023)*. USENIX Association, Anaheim, CA, USA, 7357–7374. https://www.usenix.org/conference/usenixsecurity23/presentation/liu-zhibo

[68] Haohui Mai, Jiacheng Zhao, Hongren Zheng, Yiyang Zhao, Zibin Liu, Mingyu Gao, Cong Wang, Huimin Cui, Xiaobing Feng, and Christos Kozyrakis. 2023. Honeycomb: Secure and efficient {GPU} executions via static validation. In *Proc. 17th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*. 155–172.

[69] Fan Mo, Hamed Haddadi, Kleomenis Katevas, Eduard Marin, Diego Perino, and Nicolas Kourtellis. 2021. PPFL: Privacy-preserving federated learning with trusted execution environments. In *Proceedings of the 19th annual international conference on mobile systems, applications, and services*. 94–108.

[70] Fan Mo, Ali Shahin Shamsabadi, Kleomenis Katevas, Soteris Demetriou, Ilias Leontiadis, Andrea Cavallaro, and Hamed Haddadi. 2020. Darknetz: towards model privacy at the edge using trusted execution environments. In *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services*. 161–174.

[71] Myungsuk Moon, Minhee Kim, Joonkyo Jung, and Dokyung Song. 2025. ASGARD: Protecting On-Device Deep Neural Networks with Virtualization-Based Trusted Execution Environments. In *Proceedings 2025 Network and Distributed System Security Symposium*.

[72] Rijoy Mukherjee and Rajat Subhra Chakraborty. 2023. Attacks on Recent DNN IP Protection Techniques and Their Mitigation. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 42, 11 (2023), 3642–3650. doi:10.1109/TCAD.2023.3272271

[73] Tilo Müller and Michael Spreitzenbarth. 2013. FROST. In *Applied Cryptography and Network Security*, Michael Jacobson, Michael Locasto, Payman Mohassel, and Reihaneh Safavi-Naini (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 373–388.

[74] Olga Ohrimenko, Felix Schuster, Cédric Fournet, Aastha Mehta, Sebastian Nowozin, Kapil Vaswani, and Manuel Costa. 2016. Oblivious {Multi-Party} machine learning on trusted processors. In *25th USENIX Security Symposium (USENIX Security 16)*. 619–636.

[75] Daryna Oliynyk, Rudolf Mayer, and Andreas Rauber. 2023. I Know What You Trained Last Summer: A Survey on Stealing Machine Learning Models and Defences. *Comput. Surveys* 55, 14s (2023), 1–41. doi:10.1145/3595292

[76] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. 2019. Knockoff Nets: Stealing Functionality of Black-Box Models. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*. Computer Vision Foundation / IEEE, 4954–4963. doi:10.1109/CVPR.2019.00509

[77] David Oygenblik, Carter Yagemann, Joseph Zhang, Arianna Mastali, Jeman Park, and Brendan Saltaformaggio. 2024. AI Psychiatry: Forensic Investigation of Deep Learning Networks in Memory Images. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, USA. https://www.usenix.org/conference/usenixsecurity24/presentation/oygenblik

[78] Qianqian Pan, Mianxiong Dong, Kaoru Ota, and Jun Wu. 2022. Device-Bind Key-Storageless Hardware AI Model IP Protection: A PUF and Permute-Diffusion Encryption-Enabled Approach. https://arxiv.org/abs/2212.11133

[79] Qi Pang, Shengyuan Hu, Wenting Zheng, and Virginia Smith. 2024. Attacking llm watermarks by exploiting their strengths. In *ICLR 2024 Workshop on Secure and Trustworthy Large Language Models*.

[80] Kartik Patwari, Syed Mahbub Hafiz, Han Wang, Houman Homayoun, Zubair Shafiq, and Chen-Nee Chuah. 2022. DNN model architecture fingerprinting attack on CPU-GPU edge devices. In *2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 337–355.

[81] Alessandro Pegoraro, Carlotta Segna, Kavita Kumari, and Ahmad-Reza Sadeghi. 2024. DeepEclipse: How to Break White-Box DNN-Watermarking Schemes. In *Proc. 33rd USENIX Secur. Symp.* USENIX Association.

[82] Gokulnath Rajendran, Debajit Basak, Suman Deb, and Anupam Chattopadhyay. 2024. Securing binarized neural networks via PUF-based key management in memristive crossbar arrays. *IEEE Embed. Syst. Lett.* 17, 1 (2024), 30–33.

[83] Adnan Siraj Rakin, Md Hafizul Islam Chowdhury, Fan Yao, and Deliang Fan. 2022. DeepSteal: Advanced Model Extractions Leveraging Efficient Weight Stealing in Memories. In *IEEE Symp. Secur. Privacy (SP)*. IEEE, San Francisco, CA, USA, 1157–1174.

[84] Ge Ren, Jun Wu, Gaolei Li, Shenghong Li, and Mohsen Guizani. 2024. Protecting Intellectual Property With Reliable Availability of Learning Models in AI-Based Cybersecurity Services. *IEEE Trans. Dependable Secur. Comput.* 21, 2 (2024), 600–617. doi:10.1109/TDSC.2022.3222972

[85] Pengcheng Ren, Chaoshun Zuo, Xiaofeng Liu, Wenrui Diao, Qingchuan Zhao, and Shanqing Guo. 2024. Demistify: Identifying on-device machine learning models stealing and reuse vulnerabilities in mobile apps. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[86] Abhishek Saini, Haolin Jiang, and Hang Liu. 2026. Vulnerabilities in Partial TEE-Shielded LLM Inference with Precomputed Noise. *ArXiv preprint* abs/2602.11088 (2026). https://arxiv.org/abs/2602.11088

[87] Masoumeh Shafieinejad, Nils Lukas, Jiaqi Wang, Xinda Li, and Florian Kerschbaum. 2021. On the Robustness of Backdoor-based Watermarking in Deep Neural Networks. In *Proc. 2021 ACM Workshop on Information Hiding and Multimedia Security* (Virtual Event, Belgium) *(IH&MMSec '21)*. Association for Computing Machinery, New York, NY, USA, 177–188. doi:10.1145/3437880.3460401

[88] Shuo Shao, Yiming Li, Hongwei Yao, Yiling He, Zhan Qin, and Kui Ren. 2024. Explanation as a watermark: Towards harmless and multi-bit model ownership verification via watermarking feature attribution. *ArXiv preprint* abs/2405.04825 (2024). https://arxiv.org/abs/2405.04825

[89] Tianxiang Shen, Ji Qi, Jianyu Jiang, Xian Wang, Siyuan Wen, Xusheng Chen, Shixiong Zhao, Sen Wang, Li Chen, Xiapu Luo, et al. 2022. {SOTER}: Guarding black-box inference for general neural networks at the edge. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 723–738.

[90] Victor Shoup. 2004. Sequences of games: a tool for taming complexity in security proofs. *cryptology eprint archive* (2004).

[91] Hemavathy Sriramulu and V. S. Kanchana Bhaaskaran. 2023. Arbiter PUF - A Review of Design, Composition, and Security Aspects. *IEEE Access* 11 (2023), 33979–34004. doi:10.1109/ACCESS.2023.3264016

[92] G. Edward Suh and Srinivas Devadas. 2007. Physical Unclonable Functions for Device Authentication and Secret Key Generation. In *Proc. 44th Design Automat. Conf. (DAC)*. IEEE, 9–14. doi:10.1145/1278480.1278484

[93] Shichang Sun, Haoqi Wang, Mingfu Xue, Yushu Zhang, Jian Wang, and Weiqiang Liu. 2021. Detect and Remove Watermark in Deep Neural Networks via Generative Adversarial Networks. In *Information Security: 24th International Conference, ISC 2021, Virtual Event, November 10–12, 2021, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 341–357. doi:10.1007/978-3-030-91356-4_18

[94] Yulian Sun, Vedant Bonde, Li Duan, and Yong Li. 2025. Obfuscation for Deep Neural Networks Against Model Extraction: Attack Taxonomy and Defense Optimization. In *Int. Conf. Appl. Cryptogr. Netw. Secur. (ACNS)*. Springer, 391–414.

[95] Zhichuang Sun, Ruimin Sun, Changming Liu, Amrita Roy Chowdhury, Long Lu, and Somesh Jha. 2023. ShadowNet: A Secure and Efficient On-device Model Inference System for Convolutional Neural Networks . In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 1596–1612. doi:10.1109/SP46215.2023.10179382

[96] Zhichuang Sun, Ruimin Sun, Long Lu, and Alan Mislove. 2021. Mind Your Weight(s): A Large-scale Study on Insufficient Machine Learning Model Protection in Mobile Apps. In *Proc. 30th USENIX Secur. Symp.* USENIX Association, Vancouver, BC, Canada, 1955–1972.

[97] Mingxing Tan and Quoc V. Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proc. of ICML (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 6105–6114. http://proceedings.mlr.press/v97/tan19a.html

[98] Fatemeh Tehranipoor, Nima Karimian, Wei Yan, and John A. Chandy. 2017. DRAM-Based Intrinsic Physically Unclonable Functions for System-Level Security and Authentication. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25, 3 (2017), 1085–1097. doi:10.1109/TVLSI.2016.2606658

[99] Jinyu Tian, Jiantao Zhou, and Jia Duan. 2021. Probabilistic Selective Encryption of Convolutional Neural Networks for Hierarchical Services. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*. Computer Vision Foundation / IEEE, 2205–2214. doi:10.1109/CVPR46437.2021.00224

[100] Philippe Tillet, Hsiang-Tsung Kung, and David D. Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, MAPL@PLDI 2019, Phoenix, AZ, USA, June 22, 2019*, Tim Mattson, Abdullah Muzahid, and Armando Solar-Lezama (Eds.). ACM, 10–19. doi:10.1145/3315508.3329973

[101] Hakan Tora, Erhan Gokcay, Mehmet Turan, and Mohamed Buker. 2022. A generalized Arnold's Cat Map transformation for image scrambling. *Multimedia Tools and Applications* 81, 22 (2022), 31349–31362. doi:10.1007/s11042-022-11985-2

[102] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. 2021. Training data-efficient image transformers & distillation through attention. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event (Proceedings of Machine Learning Research)*, Marina Meila and Tong Zhang (Eds.). PMLR, 10347–10357. http://proceedings.mlr.press/v139/touvron21a.html

[103] Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. 2016. Stealing Machine Learning Models via Prediction APIs. In *25th USENIX Security Symposium (USENIX Security 16)*. USENIX Association, Austin, TX, 601–618. https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/tramer

[104] Yusuke Uchida, Yuki Nagai, Shigeyuki Sakazawa, and Shin'ichi Satoh. 2017. Embedding Watermarks into Deep Neural Networks. In *ACM Int. Conf. Multimedia Retrieval (ICMR)*. ACM, 269–277. doi:10.1145/3078971.3078974

[105] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[106] Stavros Volos, Kapil Vaswani, and Rodrigo Bruno. 2018. Graviton: Trusted execution environments on {GPUs}. In *Proc. 13th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*. 681–696.

[107] Jinwen Wang, Yuhao Wu, Han Liu, Bo Yuan, Roger Chamberlain, and Ning Zhang. 2023. IP Protection in TinyML. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, San Francisco, CA, USA, 1–6. doi:10.1109/DAC56929.2023.10247898

[108] Pengli Wang, Bingyou Dong, Yifeng Cai, Zheng Zhang, Junlin Liu, Huanran Xue, Ye Wu, Yao Zhang, and Ziqi Zhang. 2025. Game of Arrows: On the (In-)Security of Weight Obfuscation for On-Device TEE-Shielded LLM Partition Algorithms. In *Proc. 34th USENIX Secur. Symp.* USENIX Association, 279–298.

[109] Siyue Wang, Xiao Wang, Pin-Yu Chen, Pu Zhao, and Xue Lin. 2021. Characteristic Examples: High-Robustness, Low-Transferability Fingerprinting of Neural Networks. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021, Virtual Event / Montreal, Canada, 19-27 August 2021*, Zhi-Hua Zhou (Ed.). ijcai.org, 575–582. doi:10.24963/IJCAI.2021/80

[110] Zihan Wang, Zhongkui Ma, Xinguo Feng, Ruoxi Sun, Hu Wang, Minhui Xue, and Guangdong Bai. 2024. CORELOCKER: Neuron-level Usage Control. In *IEEE Symp. Secur. Privacy (SP)*. IEEE, 2497–2514. doi:10.1109/SP54263.2024.00233

[111] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. 2023. Transformers in Time Series: A Survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th-25th August 2023, Macao, SAR, China*. ijcai.org, 6778–6786. doi:10.24963/IJCAI.2023/759

[112] Xiaolong Wu, Dave (Jing) Tian, and Chung Hwan Kim. 2023. Building GPU TEEs using CPU Secure Enclaves with GEVisor. In *Proc. 14th ACM Symp. Cloud Comput. (SOCC)*. Santa Cruz, CA. doi:10.1145/3620678.3624659

[113] Yue Wu, Zhongyun Hua, and Yicong Zhou. 2015. n-dimensional discrete cat map generation using Laplace Expansions. *IEEE transactions on cybernetics* 46, 11 (2015), 2622–2633.

[114] Gaojian Xiong, Yu Sun, Jianhua Liu, Jian Cui, and Jianwei Liu. 2025. LoRO: Real-Time on-Device Secure Inference for LLMs via TEE-Based Low Rank Obfuscation. In *39th Annu. Conf. Neural Inf. Process. Syst.*

[115] Mingfu Xue, Zhiyu Wu, Yushu Zhang, Jian Wang, and Weiqiang Liu. 2023. AdvParams: An Active DNN Intellectual Property Protection Technique via Adversarial Perturbation Based Parameter Encryption. *IEEE Trans. Emerg. Top. Comput.* 11, 3 (2023), 664–678. doi:10.1109/TETC.2022.3231012

[116] Anli Yan, Huali Ren, Kanghua Mo, Zhenxin Zhang, Shaowei Wang, and Jin Li. 2025. Enhancing Model Intellectual Property Protection with Robustness Fingerprint Technology. *IEEE Trans. Inf. Forensics Secur.* 20 (2025), 9235–9249. doi:10.1109/TIFS.2025.3602244

[117] Yifan Yan, Xudong Pan, Mi Zhang, and Min Yang. 2023. Rethinking White-Box Watermarks on Deep Learning Models under Neural Structural Obfuscation. In *Proc. 32nd USENIX Secur. Symp.* USENIX Association, 2347–2364.

[118] Yifan Yan, Xudong Pan, Mi Zhang, and Min Yang. 2023. Rethinking White-Box Watermarks on Deep Learning Models under Neural Structural Obfuscation. In *Proc. the 32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, USA, 2347–2364. https://www.usenix.org/conference/usenixsecurity23/presentation/yan

[119] Yuval Yarom and Katrina Falkner. 2014. Flush+Reload: A High Resolution, Low Noise, L3 Cache Side-Channel Attack. In *23rd USENIX Security Symposium (USENIX Security 14)*. USENIX Association, San Diego, CA, 719–732. https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/yarom

[120] Yuanyuan Yuan, Zhibo Liu, Sen Deng, Yanzuo Chen, Shuai Wang, Yinqian Zhang, and Zhendong Su. 2024. CipherSteal: Stealing Input Data from TEE-Shielded Neural Networks with Ciphertext Side Channels. In *Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, San Francisco, CA, USA, 79–79. https://www.ieee-security.org/TC/SP2025/

[121] Yuanyuan Yuan, Zhibo Liu, Sen Deng, Yanzuo Chen, Shuai Wang, Yinqian Zhang, and Zhendong Su. 2024. Hypertheft: Thieving model weights from TEE-shielded neural networks via ciphertext side channels. In *Proc. 2024 ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*. 4346–4360.

[122] Jialong Zhang, Zhongshu Gu, Jiyong Jang, Hui Wu, Marc Ph Stoecklin, Heqing Huang, and Ian Molloy. 2018. Protecting intellectual property of deep neural networks with watermarking. In *Proceedings of the 2018 on Asia conference on computer and communications security*. 159–172.

[123] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, Chong Wang, and Xin Peng. 2025. Killing two birds with one stone: Malicious package detection in NPM and PyPI using a single model of malicious behavior sequence. *ACM Trans. Softw. Eng. Methodol.* 34, 4 (2025), 1–28.

[124] Xiaohan Zhang, Jinwen Wang, Yueqiang Cheng, Qi Li, Kun Sun, Yao Zheng, Ning Zhang, and Xinghua Li. 2024. Interface-Based Side Channel in TEE-Assisted Networked Services. *IEEE/ACM Transactions on Networking* 32, 1 (2024), 613–626. doi:10.1109/TNET.2023.3294019

[125] Yinxing Zhang, Zhongyun Hua, Han Bao, Hejiao Huang, and Yicong Zhou. 2022. An n-dimensional chaotic system generation method using parametric pascal matrix. *IEEE Trans. Ind. Inform.* 18, 12 (2022), 8434–8444.

[126] Yicheng Zhang, Rozhin Yasaei, Hao Chen, Zhou Li, and Mohammad Abdullah Al Faruque. 2021. Stealing neural network structure through remote FPGA side-channel analysis. *IEEE Trans. Inf. Forensics Secur.* 16 (2021), 4377–4388.

[127] Ziqi Zhang, Chen Gong, Yifeng Cai, Yuanyuan Yuan, Bingyan Liu, Ding Li, Yao Guo, and Xiangqun Chen. 2024. No Privacy Left Outside: On the (In-)Security of TEE-Shielded DNN Partition for On-Device ML. In *2024 IEEE Symp. Secur. Privacy (SP)*. IEEE Computer Society, Washington, DC, USA, 3327–3345. doi:10.1109/SP54263.2024.00052

[128] Jingjing Zhao, Qingyue Hu, Gaoyang Liu, Xiaoqiang Ma, Fei Chen, and Mohammad Mehedi Hassan. 2020. AFA: Adversarial fingerprinting authentication for deep neural networks. *Computer Communications* 150 (2020), 488–497.

[129] Yue Zheng, Si Wang, and Chip-Hong Chang. 2022. A DNN fingerprint for non-repudiable model ownership identification and piracy detection. *IEEE Trans. Inf. Forensics Secur.* 17 (2022), 2977–2989.

[130] Tong Zhou, Yukui Luo, Shaolei Ren, and Xiaolin Xu. 2023. NNSplitter: An Active Defense Solution for DNN Model via Automated Weight Obfuscation. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 42614–42624. https://proceedings.mlr.press/v202/zhou23h.html

[131] Xirong Zhuang, Lan Zhang, Chen Tang, Huiqi Liu, Bin Wang, Yan Zheng, and Bo Ren. 2023. DeepContract: Controllable Authorization of Deep Learning Models. In *Proceedings of the 39th Annual Computer Security Applications Conference* (Austin, TX, USA) *(ACSAC '23)*. Association for Computing Machinery, New York, NY, USA, 609–620. doi:10.1145/3627106.3627107